

MAD-OPD: Breaking the Ceiling in On-Policy Distillation via Multi-Agent Debate

Jianze Wang^{1,2,*}, Ying Liu², Jinlong Chen², Xuchun Hu², Qilong Zhang²,
Yu Cao², Jun Wang², Hua Yang², Yong Xie^{1,†}, Qianglong Chen^{2,†}

¹School of Artificial Intelligence and Automation, Huazhong University of Science and Technology

²Alibaba Group

✉ swordwang@hust.edu.cn, qianglong.cql@alibaba-inc.com

Abstract

On-policy distillation (OPD) trains a student on its own trajectories under token-level teacher supervision, but existing methods are capped by a *single-teacher capability ceiling*: when the teacher errs, the student inherits the error. OPD also remains largely unexplored in agentic tasks, where per-step errors compound across long trajectories and destabilize training. We propose **MAD-OPD (Multi-Agent Debate-driven On-Policy Distillation)**, which breaks this ceiling by recasting the distillation teacher as a deliberative collective of teachers that debate over the student’s on-policy state; the debate produces an *emergent collective intelligence* that supplies token-level supervision, with each teacher’s contribution weighted by its post-debate confidence. To extend OPD to agentic tasks, we also introduce On-Policy Agentic Distillation (OPAD), which adds step-level sampling to stabilize training under multi-step error compounding. We additionally derive a task-adaptive divergence principle, selecting JSD (Jensen–Shannon divergence) for agentic stability and reverse KL (Kullback–Leibler) divergence for code generation, and verify it both theoretically and empirically. Across six teacher–student configurations (Qwen3 and Qwen3.5; 1.7B–14B students, 8B–32B teachers) and five agentic and code benchmarks, MAD-OPD ranks first across all six configurations; on the 14B+8B→4B setting it lifts the agentic average by +2.4% and the code average by +3.7% over the stronger single-teacher OPD. Code is available at: <https://github.com/chiefovoavicii/MAD-OPD>.

1 Introduction

On-policy distillation (OPD) has rapidly become a standard post-training recipe for modern LLMs [28], adopted in the strong-to-weak distillation of Qwen3 [37] and the multi-teacher OPD post-training of DeepSeek-V4 [10]; it offers dense on-policy supervision as an efficient alternative to outcome-reward reinforcement learning (RL) [31] and off-policy sequence-level distillation [21]. Recent work extends OPD with privileged information [41], reward extrapolation [39], verifiable rewards [38], and context internalization [40]; see Song et al. [33] for a survey. However, three limitations remain. **(L1) Single-teacher capability ceiling.** Existing OPD methods distill from a single teacher, upper-bounding the student by that teacher’s capability; diagnostic work further shows that even a stronger teacher can fail to lift the student [22]. **(L2) Narrow task coverage.** Current OPD research concentrates on math and commonsense reasoning [1, 13], leaving *agentic* domains [35] (multi-step tool use with environment feedback) largely unexplored: per-step errors compound across long trajectories [12, 22]. **(L3) Ad-hoc divergence choice.** OPD is mathematically equivalent to dense token-level RL in which the divergence D directly shapes the reward signal [39]; yet recent

*Work done during research internship at Alibaba Group.

†Corresponding author.

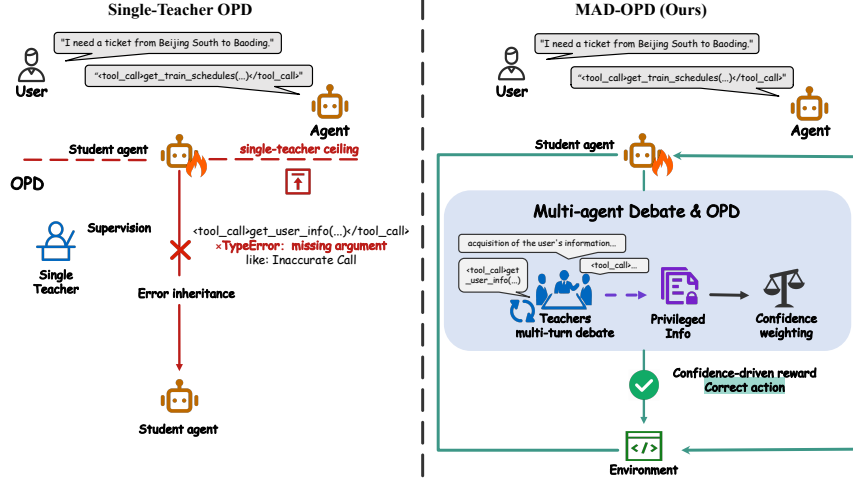


Figure 1: **Left:** a single teacher’s erroneous tool call is faithfully inherited by the student (the *single-teacher capability ceiling*). **Right:** MAD-OPD’s multi-round debate corrects each teacher’s blind spots, producing supervision that outperforms individual teachers. A worked-out instance with the full debate trajectory is in App. E.

attempts to stabilize this objective [12, 19, 20] each patch one failure mode without a principled rule linking D to task structure, leaving divergence a task-agnostic hyperparameter.

Resolving these limitations requires looking beyond a single model. Multi-teacher distillation [5] and broader multi-agent systems [8] show that collaboration among LLMs can solve problems no individual agent can; in particular, multi-agent debate (MAD) [11] produces an *emergent collective intelligence* through iterative argumentation, and Chen et al. [6] demonstrates that confidence-weighted consensus among diverse LLMs can match or exceed the strongest individual model. Yet existing multi-teacher distillation remains *off-policy*: the student passively consumes pre-computed teacher signals rather than benefiting from live teacher reactions to its own rollouts.

We propose MAD-OPD (**M**ulti-**A**gent **D**ebate-driven **O**n-**P**olicy **D**istillation; Figure 1), the first framework to bring multi-agent debate into the OPD training loop as a source of debate-conditioned teacher distributions, transferring MAD’s emergent collective intelligence to the student and breaking the single-teacher capability ceiling (**L1**). At every decision point, multiple teachers debate over the student’s on-policy state, and the debate transcript provides token-level supervision with each teacher’s contribution adaptively weighted by its post-debate confidence. To address **L2**, we propose *On-Policy Agentic Distillation* (OPAD), a distillation method dedicated to agentic tasks that brings step-level environment interaction to OPD and is compatible with both single-teacher OPD and our MAD-OPD. To address **L3**, we cast divergence selection as reward design and derive a task-adaptive divergence principle: bounded JSD for agentic stability and reverse KL for coherent code generation, both verified theoretically and empirically (Sec. 5.4).

Our contributions are summarized as follows:

- **Task-adaptive divergence principle.** Through theoretical analysis we derive a task-adaptive principle that selects JSD for agentic OPD and reverse KL for code generation.
- **Multi-Agent Debate-driven OPD framework.** We introduce MAD-OPD, the first framework to use multi-agent debate as the source of token-level on-policy supervision, transferring MAD’s emergent collective intelligence to the student and breaking the single-teacher capability ceiling.
- **OPAD for agentic tasks.** We propose *On-Policy Agentic Distillation* (OPAD), which uses step-level sampling to stabilize OPD training under multi-step error compounding, extending OPD to an agentic regime rarely addressed by prior work.

2 Related Work

On-Policy Distillation. Knowledge distillation [16] transfers a teacher’s soft predictions to a smaller student. Agarwal et al. [1], Tan et al. [34] introduced OPD as a dense on-policy alternative to

off-policy sequence-level distillation [21], and OPD has since been adopted in frontier-model post-training [10, 37] and surveyed by Song et al. [33]. Recent diagnostic work investigates when OPD fails [12, 22], and Jang et al. [19] and Jin et al. [20] propose objective-level reformulations to stabilize the reverse-KL objective. Penaloza et al. [30] extend OPD to leverage *privileged information* that is visible only to a single teacher at training time. These methods all rely on a single teacher with task-agnostic divergence; MAD-OPD instead provides debate-generated privileged information and a task-adaptive divergence.

Multi-Agent Systems. Multi-agent systems (MAS) coordinate specialized agents to solve complex tasks; Chain-of-Agents [29] distills an entire MAS pipeline into a single foundation model. The MAS instantiation closest to our setting is multi-agent debate (MAD), where iterative argumentation among agents yields an *emergent collective intelligence* that improves reasoning beyond any individual agent’s capability [6, 9, 11, 23]. Existing MAD methods, however, consume debate outputs only at inference time; MAD-OPD brings debate inside the on-policy training loop as token-level supervision.

Multi-Teacher Distillation. Multi-teacher distillation pools complementary signals from several teachers. Classical recipes aggregate teacher outputs with adaptive per-input weighting [7, 21], while LLM-era variants distill multi-agent interaction graphs or debate traces into the student [2, 5, 32, 42]. These methods all lack inter-teacher interaction: teachers produce outputs independently and the student learns from a fixed aggregate. MAD-OPD adds on-policy inter-teacher debate as the supervision source.

3 Preliminaries and Divergence Analysis

3.1 On-Policy Distillation as Reward Design

OPD as dense token-level RL. Given a teacher π^* and a student π_θ with parameters θ , OPD minimizes a divergence along the student’s own trajectories [1, 33]. For input x from data distribution $\mathcal{D}$ and $\hat{y} \sim \pi_\theta(\cdot|x)$:

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{x, \hat{y} \sim \pi_\theta} \left[\frac{1}{|\hat{y}|} \sum_{t=1}^{|\hat{y}|} D(\pi^*(\cdot|x, c, \hat{y}_{<t}) \parallel \pi_\theta(\cdot|x, \hat{y}_{<t})) \right], \quad (1)$$

where D is a divergence and c denotes *privileged information* available only to the teacher [41]; in our setting, c is the multi-agent debate transcript. Hübner et al. [17], Lu and Thinking Machines Lab [28], Yang et al. [39] show that Eq. 1 corresponds to a dense token-level RL surrogate under teacher-forcing (App. D.1), where state $s_t = (x, \hat{y}_{<t})$, action $a_t = \hat{y}_t$, and per-token reward $r_D(s_t) \triangleq -D(\pi^*(\cdot|x, c, \hat{y}_{<t}) \parallel \pi_\theta(\cdot|x, \hat{y}_{<t}))$. Since this reward is entirely determined by D , **choosing a divergence is choosing the reward function.**

Notation. For brevity, when the position t is clear from context we write $p \triangleq \pi^*(\cdot|x, c, \hat{y}_{<t})$ for the privileged teacher distribution and $q \triangleq \pi_\theta(\cdot|x, \hat{y}_{<t})$ for the student distribution over the vocabulary $\mathcal{V}$. The student’s pre-softmax logit at coordinate i is z_i , so $q(v) = \text{softmax}(z)_v$. All logarithms are natural (base e).

The privileged p - q gap. Privileged distillation creates a structural asymmetry: p conditions on c but q does not, so p and q can disagree sharply at student-visited states. The asymmetry takes two forms: (a) the teacher assigns *near-zero* probability to tokens the student samples ($p(v) \rightarrow 0$ while $q(v) > 0$), and (b) the teacher concentrates on *multiple* valid tokens that the student must choose between. The first form dominates multi-step agentic tasks; the second dominates code generation.

3.2 Theoretical Properties of Divergences

Three divergences are standard candidates for D in OPD:

$$\text{Forward KL: } D_{\text{KL}}^{\rightarrow}(p\|q) = \sum_v p(v) \log \frac{p(v)}{q(v)}, \quad (2)$$

$$\text{Reverse KL: } D_{\text{KL}}^{\leftarrow}(p\|q) \triangleq D_{\text{KL}}(q\|p) = \sum_v q(v) \log \frac{q(v)}{p(v)}, \quad (3)$$

$$\text{JSD: } \text{JSD}_{\beta}(p\|q) = \beta D_{\text{KL}}(p\|m) + (1-\beta) D_{\text{KL}}(q\|m), \quad m = \beta p + (1-\beta)q. \quad (4)$$

The two lemmas below characterize each divergence along two axes: *geometry* (mode-covering vs. mode-seeking) and *per-token logit-gradient boundedness*. We use $\beta = 0.5$ throughout: the canonical symmetric form attaining the maximum loss bound $\log 2$ (Lemma 1.1), under which our gradient analysis is tightest.

Lemma 1 (JSD: bounded loss and bounded logit gradient [24]). *For any distributions p, q over $\mathcal{V}$ and $\beta \in (0, 1)$:*

1. **Loss bound:** $\text{JSD}_{\beta}(p\|q) \in [0, H(\beta)]$, where $H(\beta) = -\beta \log \beta - (1-\beta) \log(1-\beta)$. For $\beta=0.5$: $\text{JSD}_{0.5} \in [0, \log 2]$.
2. **Logit-gradient bound:** for the symmetric case $\beta = 0.5$ used throughout this paper, the gradient with respect to student logits is uniformly bounded, $\|\nabla_z \text{JSD}_{0.5}(p\|q)\|_{\infty} \leq 2$, regardless of the support overlap between p and q . The bound holds in the worst case with no independence or distributional assumption, and extends to any β bounded away from $\{0, 1\}$ with a β -dependent constant (App. D.2).

Proof in App. D.2.

Lemma 2 (Reverse KL: mode concentration [13]). *Let $p = \sum_{j=1}^J \alpha_j p_j$ be a mixture with J modes having disjoint supports. Under the softmax parameterization, gradient-descent on $D_{\text{KL}}(q\|p)$ converges to a stationary point q^* that concentrates on the dominant mode p_{j^*} ($\alpha_{j^*} = \max_j \alpha_j$), with cost $D_{\text{KL}}(q^*\|p) = -\log \alpha_{j^*}$ [13].*

Proof in App. D.3.

3.3 Task-Adaptive Divergence Principle

We now translate the lemmas into task-specific propositions for the two regimes of §3.1, which demand opposite divergences.

Agentic tasks: gradient stability under privileged p - q gaps. A multi-step agentic OPD trajectory $\tau = (a_1, o_1, \dots, a_M, o_M)$ accumulates errors over M steps, where a mistake at step m cascades to every subsequent decision. The privileged p - q gap (§3.1) takes its sharpest form here: under the dense OPD estimator (App. D.1), what propagates through training is not the trajectory *return* but the per-step *logit gradient* $\nabla_{z_t} D$.

Proposition 1 (Per-token gradient stability under privileged p - q gaps). *Under the dense OPD gradient estimator, the per-token logit gradient satisfies:*

1. **JSD:** $\|\nabla_{z_t} \text{JSD}_{\beta}(p\|q)\|_{\infty} \leq 2$ worst-case (Lemma 1.2), with no support-overlap or independence assumption. By per-position decoupling, $\|\nabla_z \mathcal{L}_{\text{JSD}}(\tau)\|_{\infty} \leq 2$ under either reduction (single-turn mean as in Eq. 1, or sum-of-per-step-means as in OPAD), independent of trajectory length M and of the teacher-student gap.
2. **Reverse KL:** $\|\nabla_{z_t} D_{\text{KL}}^{\leftarrow}(p\|q)\|_{\infty}$ is unbounded: it contains a term proportional to $q(i) \log(q(i)/p(i))$, which diverges whenever $p(i) \rightarrow 0$ with $q(i) > 0$, the precise regime created by privileged supervision. Hence $\|\nabla_z \mathcal{L}_{\text{KL}}^{\leftarrow}(\tau)\|_{\infty}$ is unbounded under any reduction.
3. **Forward KL:** $\|\nabla_{z_t} D_{\text{KL}}^{\rightarrow}(p\|q)\|_{\infty} = \|q - p\|_{\infty} \leq 1$ is bounded; mode-covering nonetheless degrades agentic performance (App. D.6).

Proof sketch. Part 1 follows from Lemma 1.2 plus per-position additive decomposition (the ℓ_{∞} norm of the trajectory gradient inherits the per-position bound). Part 2 follows from the softmax-Jacobian term $q(i) \log(q(i)/p(i))$ diverging as $p(i) \rightarrow 0$ with $q(i) > 0$. Part 3 from $|q(i) - p(i)| \leq 1$. Full derivations in App. D.4.

Code generation: mode concentration on coherent paths. For code, multiple correct implementations (different algorithms, naming, orderings) make the teacher distribution a mixture $p = \sum_{j=1}^J \alpha_j p_j$ over J disjoint code paths.

Proposition 2 (Reverse KL produces coherent code, forward KL and JSD do not). *Let $p = \sum_{j=1}^J \alpha_j p_j$ be a per-token multi-modal code distribution with disjoint-support modes and a path-coherent dominant mode j^* (consistent across positions). For a softmax-parameterized student q^* trained against each divergence:*

1. **Reverse KL:** q^* concentrates on the dominant mode p_{j^*} ($j^* = \arg \max_j \alpha_j$), assigning near-zero mass to other modes and avoiding incoherent splicing.
2. **Forward KL:** q^* covers all modes ($q^*(v) > 0$ on $\bigcup_j \text{supp}(p_j)$), averaging incompatible code structures.
3. **JSD:** q^* partially concentrates on p_{j^*} but retains non-negligible mass on secondary modes.

Proof sketch. Part 1 follows from Lemma 2: the infinite penalty on inter-mode mass drives q^* onto a single mode. Part 2 follows from $D_{\text{KL}}^{\rightarrow}(p||q) = \infty$ whenever $q(v) = 0$ on $\text{supp}(p)$. Part 3 follows from JSD’s mixture $m \geq \beta p$, which finitely penalizes inter-mode mass. Full derivations in App. D.5.

Combining Propositions 1 and 2 yields the selection principle that determines which divergence each task structure demands.

Remark 1 (Task-adaptive divergence selection principle). *For OPD with a privileged teacher, the divergence D best matched to the task structure is selected as follows:*

- **Multi-step agentic tasks** $\rightarrow D = \text{JSD}_{\beta}$. By Prop. 1.1, the per-position logit gradient is bounded by 2 uniformly across positions and privileged p - q gaps, supporting stable long-trajectory training (App. C.5). Reverse KL admits no such finite bound under any reduction (Prop. 1.2); forward KL is bounded but mode-covering produces incoherent rollouts (Prop. 1.3 and App. D.6).
- **Code generation** $\rightarrow D = D_{\text{KL}}^{\leftarrow}$. By Prop. 2.1, the reverse-KL minimizer concentrates probability mass on a single coherent implementation path; forward KL and JSD provably splice incompatible modes (Prop. 2.2–3).

Sec. 5.4 validates both predictions empirically (Table 4, Figure 6).

4 Method: MAD-OPD

MAD-OPD translates the task-adaptive divergence principle of Sec. 3.3 into a concrete training pipeline (Figure 2). Sec. 4.1 produces the privileged information c that Eq. 1 requires via inter-teacher debate; Sec. 4.2 aggregates teachers by post-debate confidence; Sec. 4.3 writes the resulting task-adaptive loss; Sec. 4.4 presents OPAD, an OPD-based training method for agentic tasks that brings step-level environment interaction.

4.1 Multi-Agent Debate as Privileged Information Generation

Given K teachers $\{T_1, \dots, T_K\}$ and the current state s_m (the prompt x for single-turn tasks, or the context $(x, \tau_{<m})$ for multi-step agentic tasks), debate runs for R rounds. At round 1 each teacher samples independently with no inter-teacher context; at every subsequent round each teacher revises its round- r response h_r^k after reading all prior responses from every teacher:

$$h_r^k \sim p_{T_k}(\cdot \mid s_m, \{h_{r'}^j\}_{j=1, \dots, K, r' < r}), \quad k = 1, \dots, K, \quad r = 1, \dots, R. \quad (5)$$

The diverse initial responses (driven by differences in capacity or parameterization) seed the deliberation, and the iterative exchange lets teachers challenge each other and catch errors any individual teacher might miss. The full debate history $\mathcal{H}_m^R = \{h_r^k\}_{k,r}$ forms the privileged context:

Definition 1 (Debate-generated privileged information). *The privileged context $c_m \triangleq \mathcal{H}_m^R$ is visible to teachers ($p_{T_k}(\cdot \mid s_m, c_m, \hat{y}_{<t})$) but hidden from the student ($p_S(\cdot \mid s_m, \hat{y}_{<t})$), instantiating Eq. 1 with collaboratively generated c .*

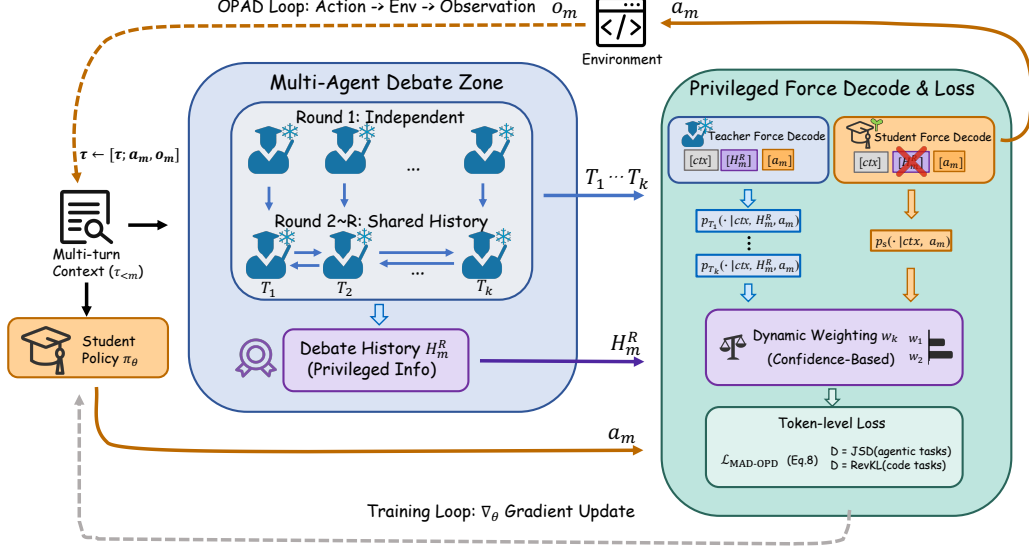


Figure 2: **The MAD-OPD Pipeline.** At trajectory step m , the student π_θ samples an on-policy action a_m ; K teachers debate for R rounds to produce a transcript $\mathcal{H}_m^R$ visible only to the teachers, establishing the privileged p - q gap. Teachers then force-decode a_m and contribute to a confidence-weighted divergence D (JSD for agents, $D_{\text{KL}}^\leftarrow$ for code); gradients update only π_θ . The dashed border marks the OPAD outer loop where the environment returns observation o_m for the next step.

4.2 Confidence-Based Dynamic Teacher Weighting

After round R , each teacher T_k self-reports a scalar confidence score $c_k \in [0, 100]$ extracted from its structured output, reflecting its certainty given the full debate history $\mathcal{H}_m^R$:

$$c_k = f_{\text{conf}}(T_k; s_m, \mathcal{H}_m^R). \quad (6)$$

Per-teacher weights are obtained by softmax normalization over the unit-scaled scores $\tilde{c}_k := c_k/100 \in [0, 1]$:

$$w_k = \frac{\exp(\tilde{c}_k/\tau_{\text{conf}})}{\sum_{j=1}^K \exp(\tilde{c}_j/\tau_{\text{conf}})}, \quad \sum_{k=1}^K w_k = 1, \quad (7)$$

with temperature $\tau_{\text{conf}} = 1.0$ throughout. Because c_k is produced *after* all R rounds, the weights reflect post-deliberation certainty rather than pre-debate agreement: a teacher whose position weakens during debate contributes less. Robustness under biased or uninformative confidence is analyzed in App. C.1.

4.3 Token-Level Distillation Objective

The right panel of Figure 2 shows the force-decode stage. Each teacher processes the student’s on-policy sample $\hat{y}$ with the debate transcript $\mathcal{H}_m^R$ appended to its context: $p_{T_k}(\cdot \mid s_m, \mathcal{H}_m^R, \hat{y}_{<t})$. The student encodes the same tokens without this transcript: $p_S(\cdot \mid s_m, \hat{y}_{<t})$. The MAD-OPD loss aggregates per-teacher divergences weighted by their confidence scores:

$$\mathcal{L}_{\text{MAD-OPD}}(\theta) = \mathbb{E}_{s_m \sim \mathcal{D}, \hat{y} \sim \pi_\theta(\cdot \mid s_m)} \left[\frac{1}{|\hat{y}|} \sum_{t=1}^{|\hat{y}|} \sum_{k=1}^K w_k \cdot D(p_{T_k}(\cdot \mid s_m, \mathcal{H}_m^R, \hat{y}_{<t}) \parallel p_S(\cdot \mid s_m, \hat{y}_{<t})) \right]. \quad (8)$$

Following Remark 1, we set $D = \text{JSD}_\beta$ for agentic tasks and $D = D_{\text{KL}}^\leftarrow$ for code generation, where $D_{\text{KL}}^\leftarrow(P \parallel Q) \triangleq D_{\text{KL}}(Q \parallel P)$ explicitly denotes the reverse KL. Divergences span the full vocabulary; teacher logits are treated as fixed targets, so gradients flow only through the student.

4.4 On-Policy Agentic Distillation (OPAD)

Guided by Prop. 1, we propose *On-Policy Agentic Distillation* (OPAD), instantiated here with MAD-OPD but compatible with single-teacher OPD. The student generates an on-policy trajectory

$\tau = (a_1, o_1, \dots, a_M, o_M)$ of length M : at step m , with state $s_m = (x, \tau_{<m})$, it samples action $a_m \sim \pi_\theta(\cdot | s_m)$ and the environment returns $o_m \sim P^{\text{env}}(\cdot | s_m, a_m)$. At each step, teachers debate on s_m and force-decode a_m conditioned on the resulting transcript $\mathcal{H}_m^R$, giving the per-step OPAD loss:

$$\mathcal{L}_D^{\text{opad}}(s_m) = \frac{1}{|a_m|} \sum_{t=1}^{|a_m|} \sum_{k=1}^K w_k \cdot D(p_{T_k}(\cdot | s_m, \mathcal{H}_m^R, a_{m,<t}) \parallel p_S(\cdot | s_m, a_{m,<t})). \quad (9)$$

The total trajectory loss sums per-step contributions over the on-policy rollout:

$$\mathcal{L}_{\text{OPAD}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \tau \sim \pi_\theta(\cdot | x)} \left[\sum_{m=1}^M \mathcal{L}_D^{\text{opad}}(s_m) \right]. \quad (10)$$

The key adaptive property: because debate happens *per step* and conditions on actual observations $\{o_1, \dots, o_{m-1}\}$ (not hypothetical ones), supervision adapts to whatever trajectory the student produces. Algorithm 1 (App. A) gives the complete procedure.

5 Experiments

We evaluate MAD-OPD across six teacher–student configurations, two model families, and five benchmarks. Four research questions guide the analysis: **(RQ1)** Does debate break the single-teacher capability ceiling? **(RQ2)** How do gains scale across teacher–student capability? **(RQ3)** Do debate and confidence weighting yield strictly non-redundant gains over naive on-policy aggregation? **(RQ4)** Do divergences behave as the theory of Sec. 3.3 predicts?

5.1 Experimental Setup

We evaluate MAD-OPD on two model families, Qwen3 [37] and Qwen3.5, across six teacher–student configurations spanning teacher-to-student size ratios from $4.7\times$ to $15.5\times$: two Qwen3 teacher pairs (14B+8B and 32B+30B-A3B, with the 30B-A3B teacher being the Qwen3-30B-A3B-Instruct-2507 release) and one Qwen3.5 pair (27B+9B), with students from 1.7B to 14B. Training data is ToolACE [26] ($\sim 16\text{K}$ step-split agentic instances) for agentic tasks and OpenThoughts3 [14] (30K problems) for code tasks. The benchmarks instantiate the agentic-vs-code split of our task-adaptive divergence principle (Sec. 3.3), namely multi-step agentic tool use (BFCL-v4 [36], τ^2 -Bench [4], VitaBench [15]) and single-turn code generation (LiveCodeBench v6 [18], MBPP+ [3, 25]); we exclude math (already covered by recent OPD work [12, 22, 38]) and focus on the less-explored agentic and code regimes that expose the divergence-selection question motivating our theory. Against this setup we benchmark four reference paradigms: the undistilled instruct model (**Base**); single-teacher OPD with the stronger teacher of each pair [1] (**OPD**); naive multi-teacher OPD without debate (**MT-OPD**, equal weights); and off-policy multi-teacher sequence distillation with a 7:3 strong-to-weak mix (**MT-SeqKD** [21]); MAD-OPD is orthogonal to RL-based methods such as GRPO [31]. All models run in non-thinking mode; hyperparameters ($K=2$, $R=2$, $\beta=0.5$), debate prompts, and full protocols are in App. B.

5.2 Main Results

(RQ1) Takeaway 1: MAD-OPD consistently outperforms the strongest single-teacher OPD across all six configurations. Table 1 shows MAD-OPD ranking first on overall Avg in every configuration, while each baseline fails on a structurally distinct axis: single-teacher OPD is capped by its teacher; MT-OPD’s per-token gradient conflict drops it below single-teacher OPD on code, because per-token averaging of two teacher distributions interpolates between distinct coherent code paths each teacher prefers and produces incoherent supervision the student then absorbs; off-policy MT-SeqKD inherits the exposure bias on-policy training avoids. MAD-OPD bypasses all three by turning inter-teacher disagreement into a debate transcript that establishes consensus *before* force-decoding, so the resulting supervision is densest where the privileged p – q gap is widest, providing a sharper signal than single-teacher argmax decoding (Prop. 1). As a concrete instance, App. C.3 reports a 4B student trained with our 14B+8B teacher debate outperforming the 14B teacher on LCB-v6 by +4.26% pass@1 and +10.29% Best-of- N (BoN@16), at competitive token cost; full BoN@16 results across all six configurations and on MBPP+ are in App. C.4.

Table 1: **Main results (%)**. Agentic tasks use $D=\text{JSD}_\beta$; code tasks use $D=D_{\text{KL}}^{\leftarrow}$ (per Remark 1). **Ag-Avg / Co-Avg**: per-task-group means. Agentic scores are accuracy; code scores are pass@1 averaged over 16 random seeds (subscript: std). **Bold**: best; underline: second best per config.

Method	Agentic Tasks ($D=\text{JSD}$)				Code Tasks ($D=D_{\text{KL}}^{\leftarrow}$)			Avg
	BFCL-v4	τ^2	Vita	Ag-Avg	LCB-v6	MBPP+	Co-Avg	
<i>Qwen3 14B+8B $\rightarrow$ 1.7B</i>								
Base	19.65	13.47	7.81	13.64	15.43 ± 0.61	51.27 ± 0.63	33.35	21.53
MT-SeqKD	15.52	13.02	9.06	12.53	15.82 ± 0.74	51.22 ± 0.75	33.52	20.93
OPD	20.91	18.72	12.16	17.26	19.26 ± 0.84	54.28 ± 0.78	36.77	25.07
MT-OPD	20.96	18.07	11.31	16.78	19.07 ± 0.73	51.85 ± 1.14	35.46	24.25
MAD-OPD	23.14	20.59	13.28	19.00	21.39 ± 1.21	55.92 ± 1.03	38.66	26.86
<i>Qwen3 14B+8B $\rightarrow$ 4B</i>								
Base	23.34	18.00	14.60	18.65	22.73 ± 0.65	60.27 ± 0.35	41.50	27.79
MT-SeqKD	22.51	26.07	15.85	21.48	23.18 ± 0.21	58.36 ± 1.32	40.77	29.19
OPD	24.79	29.17	15.81	23.26	25.00 ± 0.52	63.82 ± 1.21	44.41	31.72
MT-OPD	25.65	29.06	17.72	24.14	23.29 ± 0.41	60.64 ± 0.72	41.97	31.27
MAD-OPD	27.08	31.46	18.53	25.69	29.83 ± 0.56	66.42 ± 0.92	48.12	34.66
<i>Qwen3 32B+30B-A3B $\rightarrow$ 4B</i>								
Base	23.34	18.00	14.60	18.65	22.73 ± 0.65	60.27 ± 0.35	41.50	27.79
MT-SeqKD	23.09	35.60	13.87	24.19	25.06 ± 0.26	63.29 ± 1.15	44.17	32.18
OPD	27.70	32.39	16.20	25.43	26.96 ± 0.41	64.43 ± 1.03	45.70	33.54
MT-OPD	24.95	34.60	17.29	25.61	27.14 ± 0.43	61.92 ± 0.85	44.53	33.18
MAD-OPD	27.76	34.62	19.53	27.30	30.42 ± 0.43	66.37 ± 0.95	48.40	35.74
<i>Qwen3 32B+30B-A3B $\rightarrow$ 8B</i>								
Base	28.94	30.08	17.16	25.39	20.93 ± 0.76	63.56 ± 0.61	42.25	32.13
MT-SeqKD	30.62	39.06	16.78	28.82	21.93 ± 0.31	66.36 ± 1.52	44.14	34.95
OPD	29.94	36.91	22.02	29.62	26.83 ± 0.34	66.64 ± 0.93	46.73	36.47
MT-OPD	29.96	38.73	22.47	30.39	26.57 ± 0.38	67.20 ± 1.31	46.89	36.99
MAD-OPD	31.39	40.21	23.11	31.57	30.71 ± 0.74	67.32 ± 1.12	49.02	38.55
<i>Qwen3 32B+30B-A3B $\rightarrow$ 14B</i>								
Base	32.46	33.70	21.70	29.29	25.57 ± 0.87	75.13 ± 0.78	50.35	37.71
MT-SeqKD	33.99	40.26	23.24	32.50	27.25 ± 1.01	75.36 ± 0.85	51.30	40.02
OPD	32.16	40.53	24.47	32.39	28.73 ± 0.84	76.73 ± 1.04	52.73	40.52
MT-OPD	30.52	42.42	25.44	32.79	29.17 ± 0.84	75.50 ± 0.62	52.34	40.61
MAD-OPD	34.35	43.59	24.84	34.26	31.36 ± 1.02	77.42 ± 1.31	54.39	42.31
<i>Qwen3.5 27B+9B $\rightarrow$ 4B</i>								
Base	35.76	42.25	9.73	29.25	21.93 ± 1.02	62.73 ± 0.98	42.33	34.48
MT-SeqKD	24.04	35.54	7.33	22.30	29.06 ± 0.62	66.73 ± 0.97	47.90	32.54
OPD	34.76	43.39	9.07	29.07	28.32 ± 1.02	65.21 ± 0.57	46.77	36.15
MT-OPD	33.98	39.53	10.49	28.00	28.12 ± 0.42	63.49 ± 0.94	45.80	35.12
MAD-OPD	36.72	44.83	10.83	30.79	30.26 ± 0.46	67.32 ± 1.15	48.79	37.99

5.3 Scaling Behavior

(RQ2) Takeaway 2: MAD-OPD generalizes across families and configurations, with gains scaling alongside teacher and student capability. Figure 3 probes scaling along three axes: teacher capability, student capability, and cross-family transfer.

Debate amplifies multi-teacher complementarity. Naive MT-OPD already exceeds single-teacher OPD on Ag-Avg in 4 of 6 configurations, showing that adding a weaker teacher’s signal helps even without debate; MAD-OPD extends this lead to all 6 configurations, so the gain reflects *debated* complementarity rather than raw ensemble capacity.

Student–teacher capability gap shapes the scaling behavior. Reading $\Delta(\text{Base})$ off Figure 3(b): an over-large teacher–student gap caps the gain (Δ rises from +5.3% at 1.7B to +6.9% at 4B under the 14B+8B teachers, since at $\sim 13\times$ ratio the student cannot sample on-policy trajectories inside the teachers’ competence regime [22]); once the student is capable enough, a stronger teacher pool injects additional usable signal (at 4B, switching from 14B+8B to 32B+30B-A3B teachers lifts MAD-OPD from 34.7% to 35.7%, dashed segment in Figure 3(a)); and on Qwen3.5 27B+9B $\rightarrow$ 4B, where baselines lift Base by at most +1.67%, MAD-OPD’s +3.51% comes from converting per-query inter-teacher disagreement into a confidence-weighted consensus that uniform multi-teacher aggregation cannot recover (MT-OPD<OPD on this configuration).

5.4 Component and Divergence Analysis

(RQ3) Takeaway 3: MAD-OPD’s components are non-redundant. Figure 4(a) (full numbers in App. C.1) shows that removing any of multi-teacher aggregation, on-policy training, debate, or confidence weighting hurts overall scores, confirming each contributes independently. The default

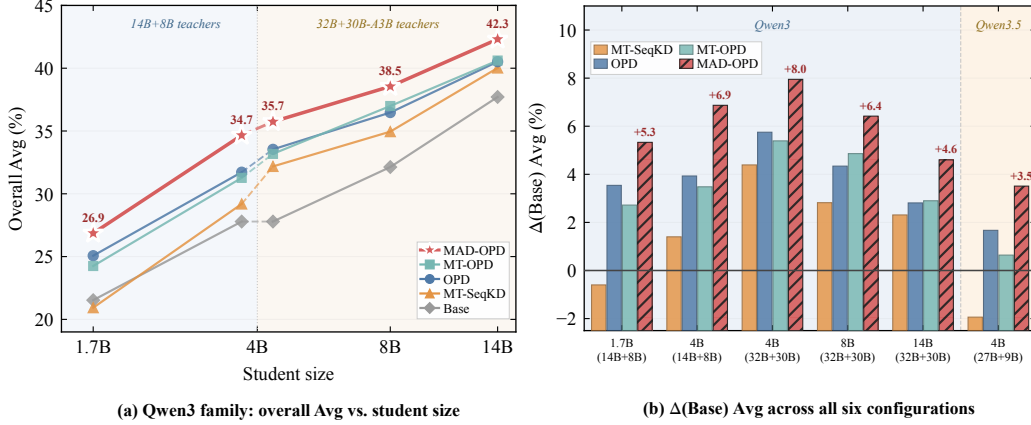


Figure 3: **Scaling behavior.** (a) Overall Avg (%) vs. student size on the Qwen3 family; solid lines connect within-pair points, dashed segments at 4B connect cross-pair points. (b) $\Delta(\text{Base})$ Avg (%), the gain over the un-distilled Base, across all six configurations.

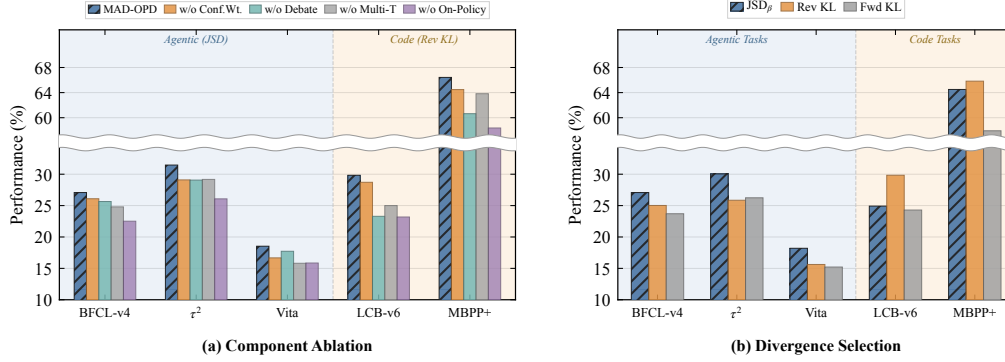


Figure 4: **Ablation study** (14B+8B $\rightarrow$ 4B). Shading separates agentic/code benchmarks. (a) Component ablation: debate adds +4.6% Co-Avg vs. MT-OPD; confidence weighting adds +1.7% Ag-Avg. (b) Per-divergence ablation (full numbers: Table 4).

$R=2$ (one inter-teacher revision round on top of independent generation) is the empirical optimum; $R=3$ overshoots due to prompt-context bloat (full $R \in \{0, 1, 2, 3\}$ sweep in App. C.6).

(RQ4) Takeaway 4: Divergence selection is dictated by task geometry, with JSD winning on agentic tasks and reverse KL on code. A controlled ablation fixing all hyperparameters except D (full numbers in App. C.2) shows the dominance pattern *inverts* along the agentic/code split: JSD leads on agentic tasks while reverse KL leads on code, with forward KL trailing on both, as Prop. 1.3 and Prop. 2.2 jointly predict (mode-covering harms both regimes; mode concentration matches code’s single-coherent-path structure). Training-dynamics evidence in Figure 6 corroborates this (App. C.5).

6 Discussion and Conclusion

We presented MAD-OPD, a multi-teacher debate framework for on-policy distillation paired with OPAD for stable multi-step agentic training, and a task-adaptive divergence principle (JSD for agentic OPD, reverse KL for code). Across six teacher–student configurations and five benchmarks, MAD-OPD ranks first in every configuration; a 4B student trained under the 14B+8B teacher debate even exceeds its 14B teacher on LCB-v6 (+4.26% pass@1, +10.29% BoN@16) at competitive token cost. This 4B-beats-14B case suggests the bottleneck of on-policy distillation is teacher-pool *diversity* rather than single-teacher capability, pointing to a complementary path to chasing larger single teachers. Future work includes richer debate structures, cross-vocabulary distillation, and extending OPAD to longer-horizon agentic environments where per-step error compounding becomes more severe.

Limitations. MAD-OPD requires teacher token-level distributions, excluding black-box (API-only) models, and assumes a shared vocabulary between teacher and student; black-box and cross-

vocabulary distillation are open extensions. Training cost also scales with $K \times R$ teacher forward passes per step.

References

- [1] Rishabh Agarwal et al. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024.
- [2] Aayush Aluru et al. SMAGDi: Socratic multi agent interaction graph distillation for efficient high accuracy reasoning. *arXiv preprint arXiv:2511.05528*, 2025.
- [3] Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [4] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [5] Justin Chen et al. MAGDi: Structured distillation of multi-agent interaction graphs improves reasoning in smaller language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [6] Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [7] Qianglong Chen, Feng Ji, Feng-Lin Li, Guohai Xu, Ming Yan, Ji Zhang, and Yin Zhang. AMTSS: An adaptive multi-teacher single-student knowledge distillation framework for multi-lingual language inference. *arXiv preprint arXiv:2305.07928*, 2023.
- [8] Yiqun Chen et al. Improving retrieval-augmented generation through multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025.
- [9] Hyeong Kyu Choi et al. Debate or vote: Which yields better decisions in multi-agent large language models? *arXiv preprint arXiv:2508.17536*, 2025.
- [10] DeepSeek-AI. DeepSeek-V4 technical report. Technical report, DeepSeek, 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.
- [11] Yilun Du et al. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- [12] Yuqian Fu, Haohuan Huang, Kaiwen Jiang, Yuanheng Zhu, and Dongbin Zhao. Revisiting on-policy distillation: Empirical failure modes and simple fixes. *arXiv preprint arXiv:2603.25562*, 2026.
- [13] Yuxian Gu et al. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.
- [14] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. OpenThoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.
- [15] Wei He, Yueqing Sun, Hongyan Hao, Xueyuan Hao, Zhikang Xia, Qi Gu, Chengcheng Han, Dengchang Zhao, Hui Su, Kefeng Zhang, et al. VitaBench: Benchmarking LLM agents with versatile interactive tasks in real-world applications. *arXiv preprint arXiv:2509.26490*, 2025.
- [16] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [17] Jonas Hübner et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- [18] Naman Jain et al. LiveCodeBench: Holistic and contamination-free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [19] Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggu Lim, and Taesup Kim. Stable on-policy distillation through adaptive target reformulation. *arXiv preprint arXiv:2601.07155*, 2026.
- [20] Woogyoul Jin, Taywon Min, Yongjin Yang, Swanand Ravindra Kadhe, Yi Zhou, Dennis Wei, Nathalie Baracaldo, and Kimin Lee. Entropy-aware on-policy distillation of language models. *arXiv preprint arXiv:2603.07079*, 2026.

- [21] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, 2016.
- [22] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026.
- [23] Tian Liang et al. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [24] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information Theory*, 37(1):145–151, 1991.
- [25] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.
- [26] Weiwen Liu et al. ToolACE: Winning the points of LLM function calling. *arXiv preprint arXiv:2409.00920*, 2024.
- [27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [28] Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026.
- [29] OPPO AI Agent Team. Chain-of-agents: End-to-end agent foundation models via multi-agent distillation and agentic RL. *arXiv preprint arXiv:2508.13167*, 2025.
- [30] Emiliano Penalosa, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged information distillation for language models. *arXiv preprint arXiv:2602.04942*, 2026.
- [31] Zhihong Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [32] Zhanming Shen et al. Merge-of-thought distillation: Distilling multi-agent reasoning into a single language model. *arXiv preprint arXiv:2509.08814*, 2025.
- [33] Mingyang Song et al. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.
- [34] Shicheng Tan, Weng Lam Tam, Yuanchun Wang, Wenwen Gong, Shu Zhao, Peng Zhang, and Jie Tang. GKD: A general knowledge distillation framework for large-scale pre-trained language model. *arXiv preprint arXiv:2306.06629*, 2023.
- [35] Zhiheng Xi et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- [36] Fanjia Yan et al. Berkeley function calling leaderboard. *arXiv preprint arXiv:2402.15671*, 2024.
- [37] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [38] Chenxu Yang et al. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*, 2026.
- [39] Wenkai Yang et al. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.
- [40] Tianzhu Ye et al. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.
- [41] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.
- [42] Xiaofeng Zhou, Heyan Huang, and Lizi Liao. Debate, reflect, and distill: Multi-agent feedback with tree-structured preference optimization for efficient language model enhancement. *arXiv preprint arXiv:2506.03541*, 2025.

A Algorithm Pseudocode

Algorithm 1 provides the complete MAD-OPD training procedure, covering both single-step (code generation, $M=1$) and multi-step OPAD (agentic tasks, $M>1$) settings. The procedure follows a *think-then-critique* paradigm: teachers first deliberate on the current state s_m to produce the privileged consensus $\mathcal{H}_m^R$, and only then evaluate the student’s already-sampled action a_m via privileged force-decoding. Teachers therefore argue about the high-level decision conditioned on s_m , not about specific tokens of the student’s draft.

Algorithm 1 MAD-OPD: Multi-Agent Debate-Driven On-Policy Distillation

Require: Teachers $\{T_1, \dots, T_K\}$, student π_θ , data $\mathcal{D}$, debate rounds R , divergence D , total steps M , learning rate η

- 1: **repeat**
- 2: Sample batch $\mathcal{B}$ from $\mathcal{D}$
- 3: **for** each $x \in \mathcal{B}$ **do**
- 4: $\mathcal{L}_x \leftarrow 0$; $\tau \leftarrow \emptyset$
- 5: **for** step $m = 1, \dots, M$ **do**
- 6: $a_m \sim \pi_\theta(\cdot \mid x, \tau)$ {on-policy; $M=1$ for non-agentic tasks}
- 7: **for** round $r = 1, \dots, R$; teacher $k = 1, \dots, K$ **in parallel do**
- 8: $h_r^k \sim p_{T_k}(\cdot \mid x, \tau, \{h_{r'}^j\}_{j,r' < r})$
- 9: **end for**
- 10: $\mathcal{H}_m^R \leftarrow \{h_r^k\}_{k,r}$ {privileged context}
- 11: $c_k \leftarrow f_{\text{conf}}(T_k; x, \tau, \mathcal{H}_m^R)$; $w_k \leftarrow \text{softmax}_k(c./ (100 \tau_{\text{conf}})) \quad \forall k$ {confidence weights}
- 12: $\mathbf{p}_{T_k}^{(t)} \leftarrow p_{T_k}(\cdot \mid x, \tau, \mathcal{H}_m^R, a_{m,<t}) \quad \forall k, t$ {force-decode **with** $\mathcal{H}_m^R$ }
- 13: $\mathbf{p}_S^{(t)} \leftarrow \pi_\theta(\cdot \mid x, \tau, a_{m,<t}) \quad \forall t$ {student **without** $\mathcal{H}_m^R$ }
- 14: $\mathcal{L}_x \leftarrow \mathcal{L}_x + \frac{1}{|a_m|} \sum_t \sum_k w_k \cdot D(\mathbf{p}_{T_k}^{(t)} \parallel \mathbf{p}_S^{(t)})$
- 15: $o_m \sim P^{\text{env}}(\cdot \mid x, \tau, a_m)$; $\tau \leftarrow \tau \oplus (a_m, o_m)$ {env step}
- 16: **end for**
- 17: **end for**
- 18: $\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{|\mathcal{B}|} \sum_x \mathcal{L}_x$ {update π_θ only}
- 19: **until** convergence

B Experimental Details

This appendix describes our complete training and evaluation setup. Sections B.1–B.7 cover the training pipeline; Sections B.8–B.12 provide per-benchmark evaluation details for the five benchmarks used in the main paper.

B.1 Training Configuration

All models operate in non-thinking mode (`enable_thinking=False`) throughout training and evaluation. Optimization uses AdamW [27] with $\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01, learning rate 1×10^{-5} with cosine decay and 5% warmup. Effective batch size is 128 with gradient clipping at norm 1.0. Agentic tasks use a maximum sequence length of 4096 tokens; code tasks allow up to 16384 tokens per generation. All configurations train for 1 epoch with checkpoints saved every 10 steps; the best checkpoint is selected by overall benchmark performance. The 30B-A3B teacher is the Qwen3-30B-A3B-Instruct-2507 release; all other Qwen3/Qwen3.5 teachers and students use the standard Instruct checkpoints of the corresponding size.

B.2 Infrastructure

All experiments run on $8 \times$ NVIDIA H20 GPUs with DeepSpeed ZeRO-3 for distributed training. Teacher models are served via two separate single-GPU processes: (i) **vLLM** generates debate-round text responses, and (ii) a **sidecar process** extracts token-level logit distributions for the distillation loss. The sidecar exposes the full vocabulary distribution rather than top- k truncation, so that JSD and reverse-KL gradients see the complete teacher density. End-to-end runtime: a single 4B-student agentic-task run completes in approximately 16 hours; code-task training takes approximately 32 hours due to the longer maximum sequence length.

B.3 Training Data

Agentic data. Training uses the ToolACE training set [26], decomposed into per-step instances under the OPAD protocol: each multi-turn trajectory is split into independent step-level training samples, producing $\sim 16\text{K}$ examples in total. At each step the student rolls out an action conditioned on the prior dialogue and tool-call history; teachers debate on the same step state and force-decode the student-sampled action. **Code data.** We sample 30K problems from OpenThoughts3 [14]. Each training example is a single on-policy completion: the student generates a code rollout, both teachers produce token-level distributions conditioned on the student’s prefix and (after debate) the debate history, and the consensus distribution serves as the training signal. Agentic and code tasks train separate checkpoints with no shared data; this matches the task-adaptive divergence design of Sec. 3.3.

B.4 Baseline Setup

Base. The undistilled instruct checkpoint of the corresponding student size, evaluated directly. **OPD** [1]. Single-teacher on-policy distillation using the stronger teacher of each pair (e.g., Qwen3-14B for the 14B+8B pair); divergence and decoding match MAD-OPD. **MT-OPD.** Naive multi-teacher OPD: both teachers produce token-level distributions independently and the loss aggregates them with uniform weights ($w_k = 1/K$); no inter-teacher debate. **MT-SeqKD** [21]. Off-policy multi-teacher sequence distillation routed by a 7:3 strong-to-weak teacher mix: the strong teacher generates 70% of the training sequences and the weak teacher 30%, mixed at the sequence level. The student is trained with sequence-level cross-entropy on the routed teacher outputs.

B.5 Debate Configuration

Each debate runs $R=2$ rounds with $K=2$ teachers. In round 1, both teachers see the task context and the student’s generated prefix and respond independently with no inter-teacher context; in round 2, each teacher additionally sees the round-1 responses (including confidence scores) prefixed with speaker identity and revises its answer. The debate temperature is 0.7 to encourage diverse initial proposals while promoting convergence. After the final round, each teacher provides a confidence score (integer in $[0, 100]$); each score is divided by 100 to yield $\tilde{c}_k \in [0, 1]$ and then softmax-normalized with temperature $\tau_{\text{conf}}=1.0$ to produce the per-teacher weights (Eq. 7). The debate prompt is kept identical across all configurations and model families. App. C.6 ablates $R \in \{0, 1, 2, 3\}$ and confirms $R=2$ as the optimal operating point.

B.6 Evaluation Protocol

All methods are trained for 1 epoch. In large-scale LLM post-training, exhaustive checkpoint selection on fully separate validation suites is often prohibitively expensive, especially for agentic evaluation (which requires user-simulator API calls) and code evaluation (which requires per-checkpoint sandbox runs). We therefore adopt a practical checkpoint-selection protocol: checkpoints are evaluated every 10 steps for agentic tasks on **BFCL-v4** and every 5 steps for code tasks on **LCB-v6**, and the highest-scoring checkpoint for each task type is then evaluated once on all five benchmarks for the final reported numbers. The same protocol is applied to all methods. Under this protocol, BFCL-v4 and LCB-v6 serve as development benchmarks for checkpoint selection, while τ^2 -Bench, VitaBench, and MBPP+ are held out from model selection; this is consistent with common practice in prior OPD work [1, 33].

Code-task pass@1 scores are averaged over 16 decoding seeds (subscript std in Tables 1 and 5); agentic-task scores are reported as a single run per the deterministic-simulator setup of each agentic benchmark, where the reported score is itself a mean over many internal subtasks or domains (8 subtasks for BFCL-v4, 3 domains $\times$ many tasks each for τ^2 -Bench and VitaBench), substantially reducing single-run variance. Following the Qwen3 technical report [37] (recommended decoding parameters for non-thinking mode), we use sampling with temperature 0.7, top- $p = 0.8$, and presence penalty 1.2 for all evaluations. Maximum generation lengths follow the defaults recommended by each benchmark: 4096 tokens for agentic tasks and 16384 tokens for code tasks. We additionally report Best-of- N (BoN@16) results ($N = 16$ samples) in App. C.4. For the multi-turn user-simulation benchmarks (τ^2 -Bench and VitaBench), we use Claude-4.6-Opus as the user simulator with a maximum of 300 environment steps per episode. Per-benchmark details (subtask coverage, license, evaluation metric) are in Sections B.8–B.12.

Table 2: Complete training and evaluation configuration.

Parameter	Agentic Tasks	Code Tasks
<i>Training</i>		
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.999$)	AdamW ($\beta_1=0.9, \beta_2=0.999$)
Weight decay	0.01	0.01
Learning rate	1×10^{-5} , cosine, 5% warmup	1×10^{-5} , cosine, 5% warmup
Effective batch size	128	128
Gradient clipping	1.0	1.0
Max sequence length	4096	16384
Data	ToolACE ($\sim 16K$, step-split)	OpenThoughts3 (30K)
Thinking mode	Non-thinking	Non-thinking
<i>Distillation</i>		
Teachers (K)	2	2
Debate rounds (R)	2	2
Debate temperature	0.7	0.7
Divergence (D)	$JSD_{0.5}$	D_{KL}^-
Logit vocabulary	Full vocabulary	Full vocabulary
Logit precision	BF16	BF16
<i>Infrastructure</i>		
Training GPUs	$8 \times$ NVIDIA H20 (ZeRO-3)	$8 \times$ NVIDIA H20 (ZeRO-3)
Teacher debate serving	vLLM (single H20)	vLLM (single H20)
Logit extraction	Sidecar (single H20)	Sidecar (single H20)
<i>Evaluation</i>		
Temperature	0.7	0.7
Top- p	0.8	0.8
Presence penalty	1.2	1.2
Max generation length	4096	16384
Decoding	Sampling	Sampling (pass@1)
BoN	–	$N=16$

B.7 Hyperparameter Summary

Table 2 consolidates all training and evaluation hyperparameters. The key task-dependent choices are the divergence function ($JSD_{0.5}$ for agentic tasks, D_{KL}^- for code) and the maximum sequence length (4096 vs. 16384 tokens). Decoding hyperparameters (temperature 0.7, top- p 0.8, presence penalty 1.2) follow the Qwen3 technical report’s recommended settings for non-thinking mode; maximum generation lengths follow each benchmark’s recommended defaults.

B.8 Benchmark: BFCL-v4

Berkeley Function-Calling Leaderboard v4 (BFCL-v4) [36] evaluates LLMs’ function-calling and tool-use capabilities through programmatically verified API invocations. The benchmark covers eight subtasks designed to stress different aspects of structured output:

- **Simple**: a single function call to a single API with explicit arguments;
- **Multiple**: selecting and calling one of several available functions based on the user query;
- **Parallel**: emitting concurrent calls to one function with different argument sets;
- **Parallel-Multiple**: emitting concurrent calls across multiple distinct functions;
- **Base**: multi-turn function calling within an ongoing dialogue (state preservation across turns);
- **Miss-Function**: the model must respond appropriately when no available function matches the user’s intent;
- **Miss-Parameter**: the model must request missing required parameters rather than hallucinating values;
- **Long-Context**: function calling under extended context length, used as a stress test of retrieval-and-call ability.

Evaluation uses AST-based correctness checking on the emitted function call schema, comparing argument names, types, and values against ground truth. The reported BFCL-v4 score in Table 1 is the unweighted mean across all eight subtasks. Available at <https://github.com/ShishirPatil/gorilla/tree/main/berkeley-function-call-leaderboard> under Apache 2.0 license.

B.9 Benchmark: τ^2 -Bench

τ^2 -**Bench** [4] extends the original τ -bench (which covered airline + retail) with a new dual-control **telecom** domain and a compositional task generator. It is a multi-turn agentic benchmark for customer-service interactions, organized into three industry domains:

- **Retail**: e-commerce account management (order returns, exchanges, refunds);
- **Airline**: flight reservation modifications, seat selection, fare changes;
- **Telecom**: account management, service plan changes, troubleshooting flows.

Each task gives the agent a domain-specific toolset (e.g., `update_order`, `modify_reservation`) and a domain policy document. The agent must execute the user’s request via tool calls while strictly adhering to the policy and tracking evolving user requirements over multiple turns. We use **Claude-4.6-Opus** as the user simulator and cap each episode at 300 environment steps. Success is graded by the official τ^2 -bench scorer (exact match of the resulting database state against the ground-truth target state). **Scoring protocol.** We follow the benchmark’s default accuracy definition: episodes that fail to execute (e.g., simulator API errors, malformed tool calls causing crashes) are excluded from the denominator, and we only score normally-executed episodes; each evaluation run is required to have $\geq 90\%$ normally-executed episodes to be counted as valid (otherwise we re-run that configuration). The reported τ^2 score in Table 1 is the pass@1 success rate averaged over the three domains. Available at <https://github.com/sierra-research/tau2-bench> under MIT license. We use the latest release of the repository, which includes 75+ task fixes for airline / retail; the telecom domain is the original τ^2 implementation. The voice and banking-knowledge modalities of the latest release are not used.

B.10 Benchmark: VitaBench

VitaBench [15] is a versatile interactive task benchmark targeting real-world life-service scenarios. It exposes 66 tools across three domains:

- **Food delivery**: restaurant search, menu navigation, order placement, delivery tracking;
- **In-store consumption**: in-shop ordering, payment, inventory checks, loyalty programs;
- **Online travel agency (OTA)**: flight/hotel/itinerary search and booking, modification, refunds.

The benchmark is constructed via a domain-policy-free framework that flexibly composes scenarios and tools, comprising 100 cross-scenario tasks (used as the main evaluation) plus 300 single-scenario tasks. Each task is derived from real user requests and requires the agent to reason across temporal and spatial dimensions, use complex tool sets, proactively clarify ambiguous instructions, and track shifting user intent throughout multi-turn conversations. Evaluation uses the official VitaBench rubric-based sliding-window scorer that supports robust assessment under stochastic interactions and diverse solution paths. We use **Claude-4.6-Opus** as the user simulator and cap each episode at 300 environment steps. **Scoring protocol.** We follow the benchmark’s default accuracy definition: episodes that fail to execute (e.g., simulator API errors, malformed tool calls causing crashes) are excluded from the denominator, and we only score normally-executed episodes; each evaluation run is required to have $\geq 90\%$ normally-executed episodes to be counted as valid (otherwise we re-run that configuration). The reported Vita score in Table 1 is the success rate averaged over the three scenarios. Available at <https://vitabench.github.io/>.

B.11 Benchmark: LiveCodeBench v6

LiveCodeBench v6 (LCB-v6) [18] is a contamination-free code generation benchmark. Problems are continuously sourced from competitive programming platforms (LeetCode, AtCoder, Codeforces) with timestamp-based filtering: each release window includes only problems published *after* the cutoff date of all evaluated models, eliminating training-data contamination by construction. Version v6 is the latest release at the time of our experiments. Each problem provides a natural-language task description and a hidden test suite (typically 20–100 hidden tests). Following the official protocol, generations are run through the hidden tests under a per-test execution timeout. We report **pass@1** (single-sample success rate, decoding at temperature 0.7 / top- p 0.8) as the primary metric and **BoN@16** (best of 16 sampled solutions per problem at temperature 0.7) as a multi-sample measure.

Table 3: **Component analysis** on MAD-OPD’s most-improved setting (14B+8B→4B). Components are added incrementally: **Multi-T** = multi-teacher aggregation; **On-Policy** = student-rollout supervision (vs. off-policy SeqKD); **Debate** = inter-teacher debate transcript as privileged information; **Conf. Wt.** = post-debate confidence-based teacher weighting (Eq. 7). **Bold** = best, underline = second-best.

Method	Multi-T	On-Policy	Debate	Conf. Wt.	Ag-Avg	Co-Avg	Avg
Base					18.65	41.50	27.79
MT-SeqKD	✓				21.48	40.77	29.19
OPD		✓			23.26	44.41	31.72
MT-OPD	✓	✓			24.14	41.97	31.27
MAD-OPD w/o Conf. Wt.	✓	✓	✓		23.95	<u>46.60</u>	<u>33.01</u>
MAD-OPD	✓	✓	✓	✓	25.69	48.12	34.66

All scores are means over 16 independent random seeds with std reported as subscript. Available at <https://github.com/LiveCodeBench/LiveCodeBench> under Apache 2.0 license.

B.12 Benchmark: MBPP+

MBPP+ [25] is the EvalPlus extension of **MBPP** [3] (Mostly Basic Python Problems). EvalPlus augments the original MBPP with extensive test cases generated through differential testing and mutation-based input synthesis (approximately $5\times$ more tests per problem on average). The augmented test suite catches code-correctness failures (e.g., off-by-one errors, edge-case mishandling) that the original MBPP tests miss, providing a substantially more reliable signal of functional correctness. The benchmark contains 378 sanitized programming problems covering basic Python data structures, algorithms, and string/list manipulation; each problem provides a natural-language description, a function signature, and the augmented test suite. We report **pass@1** and **BoN@16** on the same protocol as LCB-v6 (16 seeds, std subscript). Available at <https://github.com/evalplus/evalplus> under Apache 2.0 license.

C Additional Experiment Results

This appendix consolidates supplementary empirical evidence; each subsection indicates which research question (Sec. 5) it supports.

C.1 Component Analysis: Full Numbers (RQ3)

Table 3 reports the per-component ablation underlying Figure 4(a) on the 14B+8B→4B configuration. Components are added incrementally, isolating the contribution of each.

Robustness of confidence weighting. The Conf. Wt. increment (+1.74% Ag-Avg, +1.52% Co-Avg over the uniform-weight ablation) does not require teachers’ self-reported confidence to be well calibrated. Two structural reasons. First, the softmax in Eq. 7 cancels any teacher-shared bias in absolute confidence scale: only the *relative* ordering of $\{c_k\}$ enters the gradient. Second, the increment requires only that relative confidence ordering correlates with relative quality on average, not that any single c_k is accurate; the empirical gain over the uniform-weight ablation indicates this correlation holds in practice across the $\sim 16K$ agentic and 30K code training instances. In the worst case where confidence becomes uninformative (uniform $\{c_k\}$), Eq. 7 degenerates to $w_k = 1/K$ and the loss reduces to MT-OPD, which itself already exceeds Base on every configuration of Table 1. A systematic per-step calibration study is left to future work.

C.2 Divergence Ablation: Full Numbers (RQ4)

Table 4 reports the full divergence ablation summarized in Sec. 5.4. We fix all hyperparameters except D on the 14B+8B→4B configuration; the JSD-agentic and Rev-KL-code rows correspond to the default MAD-OPD configuration and therefore match the main MAD-OPD row in Table 1, while the off-diagonal rows isolate each non-default D . The dominance pattern inverts along the agentic/code split, as predicted by Remark 1.

Table 4: **Divergence ablation** on MAD-OPD (14B+8B→4B). JSD leads on agentic tasks; reverse KL leads on code, validating the task-dependent selection (Remark 1). Code metrics report pass@1 averaged over 16 seeds (subscript: std), matching the main-table protocol; the JSD-agentic and Rev-KL-code rows correspond to the default MAD-OPD configuration in Table 1.

	Agentic ($D=JSD$)				Code ($D=D_{KL}^*$)		
Divergence D	BFCL-v4	τ^2	Vita	Ag-Avg	LCB-v6	MBPP+	Co-Avg
Fwd KL ($D_{KL}^{\rightarrow}$)	23.70	<u>26.24</u>	15.21	21.72	24.29 ± 0.73	57.03 ± 1.13	40.66
Rev KL ($D_{KL}^{\leftarrow}$)	<u>25.02</u>	25.85	<u>15.62</u>	<u>22.16</u>	29.83 ± 0.56	66.42 ± 0.92	48.12
JSD (JSD_{β})	27.08	31.46	18.53	25.69	<u>24.92</u> ± 0.92	<u>64.50</u> ± 0.95	<u>44.71</u>

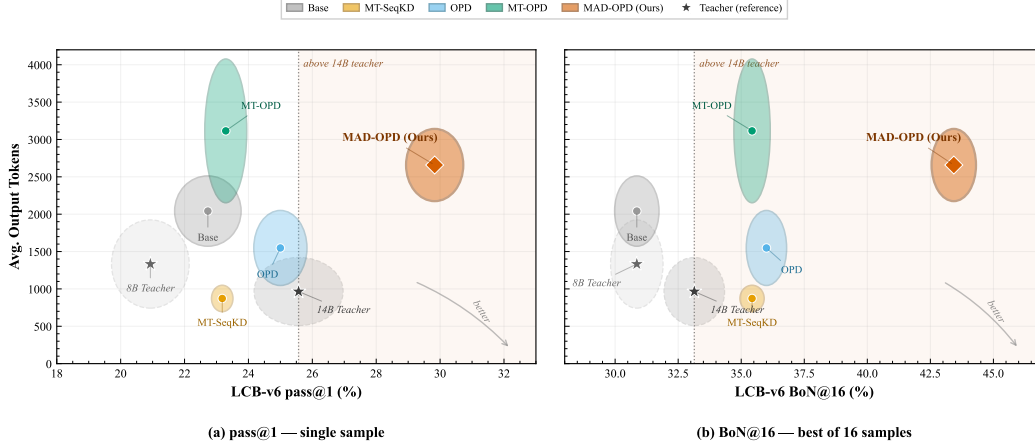


Figure 5: **Accuracy–efficiency on LCB-v6 (14B+8B→4B, 16 seeds)**. Both panels share the y-axis (avg. output tokens, lower is better) for direct token-cost comparison. Each ellipse spans ± 1 standard deviation; markers are seed means; teacher reference points (gray stars) are dashed. The vertical dotted line and shaded region mark the *14B-teacher ceiling*. **(a) pass@1 (single sample)**: only MAD-OPD crosses the 25.57% ceiling, reaching 29.83% (+4.26% over the strongest teacher, +4.83% over OPD). **(b) BoN@16 (best of 16 samples)**: MAD-OPD again clears the 33.14% teacher ceiling, reaching 43.43% (+10.29% over the 14B teacher, +7.43% over OPD); the dominance is more pronounced under multi-sample selection. In both panels, token cost is on par with MT-OPD and below MT-SeqKD’s regime, ruling out token inflation as the source of the gain.

C.3 Student Outperforms Teachers on LCB-v6 (RQ1)

Figure 5 reports an accuracy–efficiency view of the 14B+8B→4B configuration on LCB-v6 (pass@1 over 16 seeds, mean $\pm$ std). We use LCB-v6 rather than MBPP+ for this efficiency view because MBPP+ tasks are short and elicit very few output tokens per problem, leaving the token-cost axis essentially flat across methods and uninformative; LCB-v6 problems generate substantially longer solutions, exposing meaningful per-method token-budget differences.

The 4B student trained with MAD-OPD reaches 29.83% pass@1, exceeding the 14B teacher (25.57%) and the 8B teacher (20.93%): an absolute gain of +4.26% over the strongest teacher despite using $3.5\times$ fewer parameters and at competitive token cost vs. all distillation baselines. The advantage extends beyond pass@1: under BoN@16 the same 4B student reaches 43.43%, outperforming the 14B teacher’s BoN@16 (33.14%) by +10.29%. No other distillation baseline crosses the 14B-teacher pass@1 ceiling: OPD lands at 25.00%, MT-OPD at 23.29%, MT-SeqKD at 23.18%. The result shows that confidence-weighted multi-teacher debate produces supervision that exceeds the strongest single teacher in its own pool, both in single-shot and multi-sample selection regimes.

C.4 Best-of-N Results for Code Benchmarks (RQ1)

In addition to pass@1, we evaluate code generation quality using the Best-of- N (BoN) protocol with $N=16$. For each problem, we sample 16 candidate solutions (temperature 0.7) and select the one that passes the most test cases. Table 5 reports BoN@16 on both LCB-v6 and MBPP+

Table 5: **Code BoN@16 results** on LCB-v6 and MBPP+ across all configurations ($N=16$ samples per problem). **Bold**: best per configuration per benchmark; underline: second best. See Table 6 for the corresponding teacher BoN@16 reference numbers.

Method	Qwen3 14B+8B		Qwen3 32B+30B-A3B			Qwen3.5 27B+9B
	→1.7B	→4B	→4B	→8B	→14B	→4B
<i>LCB-v6 BoN@16</i>						
MT-SeqKD	<u>28.00</u>	35.43	36.57	40.57	42.86	49.71
OPD	31.43	<u>36.00</u>	36.00	36.00	38.92	41.14
MT-OPD	26.29	35.43	<u>38.86</u>	<u>39.43</u>	<u>47.43</u>	<u>49.41</u>
MAD-OPD	31.43	43.43	43.43	<u>39.43</u>	49.71	49.71
<i>MBPP+ BoN@16</i>						
MT-SeqKD	61.90	70.37	71.62	69.58	77.78	66.67
OPD	<u>66.67</u>	<u>73.81</u>	67.73	73.54	<u>80.95</u>	75.13
MT-OPD	65.18	67.72	69.58	<u>75.40</u>	80.42	<u>81.48</u>
MAD-OPD	67.73	74.34	73.81	76.46	81.69	81.69

across all six configurations. On *LCB-v6*, MAD-OPD achieves the best (or tied-best) BoN@16 in 5 of 6 configurations; MT-SeqKD leads only on 32B+30B→8B (by a 1.14-point margin) and ties with MAD-OPD on Qwen3.5 27B+9B→4B, likely because its off-policy training produces more diverse outputs that benefit from best-of- N selection. On *MBPP+*, MAD-OPD ranks first in every configuration.

C.5 Training Loss Stability under Different Divergences (RQ4)

Figure 6 visualizes the per-step training loss for MAD-OPD and OPD on agentic and code tasks under each of the three divergences (Forward KL, JSD, Reverse KL). **Agentic**. Under reverse KL, $\mathcal{L}_{\text{MAD}}$ swings across two orders of magnitude: the empirical signature of the unbounded per-token logit gradient derived in Prop. 1.2, which fires whenever the privileged teacher disprefers a student-sampled token. JSD stays bounded throughout, consistent with the per-position gradient bound of Prop. 1.1. **Code**. The dynamics invert: reverse KL converges smoothly because mode concentration (Lemma 2, Prop. 2.1) matches the single-mode structure that pass@1 rewards, whereas JSD’s interpolating geometry retains mass on secondary modes (Prop. 2.3) and produces incoherent splices across implementations.

C.6 Effect of Debate Rounds R (RQ3)

Figure 7 ablates the number of debate rounds $R \in \{0, 1, 2, 3\}$ on the 14B+8B→4B configuration across all five benchmarks. $R=0$ removes teacher generation entirely (single-teacher OPD); $R=1$ uses only the independent round and recovers MT-OPD under uniform weights; $R \geq 2$ activates inter-teacher revision. $R=2$ emerges as the empirical optimum on every benchmark, while $R=3$ overshoots: the extra round inflates the prompt context and introduces noise that erodes the gains (Ag-Avg drops by -8.27% relative to $R=2$).

C.7 Teacher Base Performance Reference

Table 6 consolidates the undistilled (Base) performance of every teacher used in the six configurations of Sec. 5.1. These numbers serve as the single-teacher upper bound that our **OPD** baseline already attains and that MAD-OPD aims to exceed via debate. The 30B-A3B teacher refers to the Qwen3-30B-A3B-Instruct-2507 release; all other teachers use the standard Instruct checkpoints of the corresponding size (App. B).

D Detailed Proofs

This appendix provides complete proofs for the lemmas and propositions in Section 3. §D.1 derives the dense OPD gradient (no REINFORCE term). §D.2–D.3 prove the per-divergence Lemmas 1 (JSD loss/gradient bounds) and 2 (reverse-KL mode concentration). §D.4 proves Proposition 1, comparing the per-token logit-gradient behaviour of all three divergences under privileged p – q gaps. §D.5 proves Proposition 2, the mode-coverage analysis for code generation. §D.6 supplements with forward-KL’s mode-covering failure mode in agentic settings.

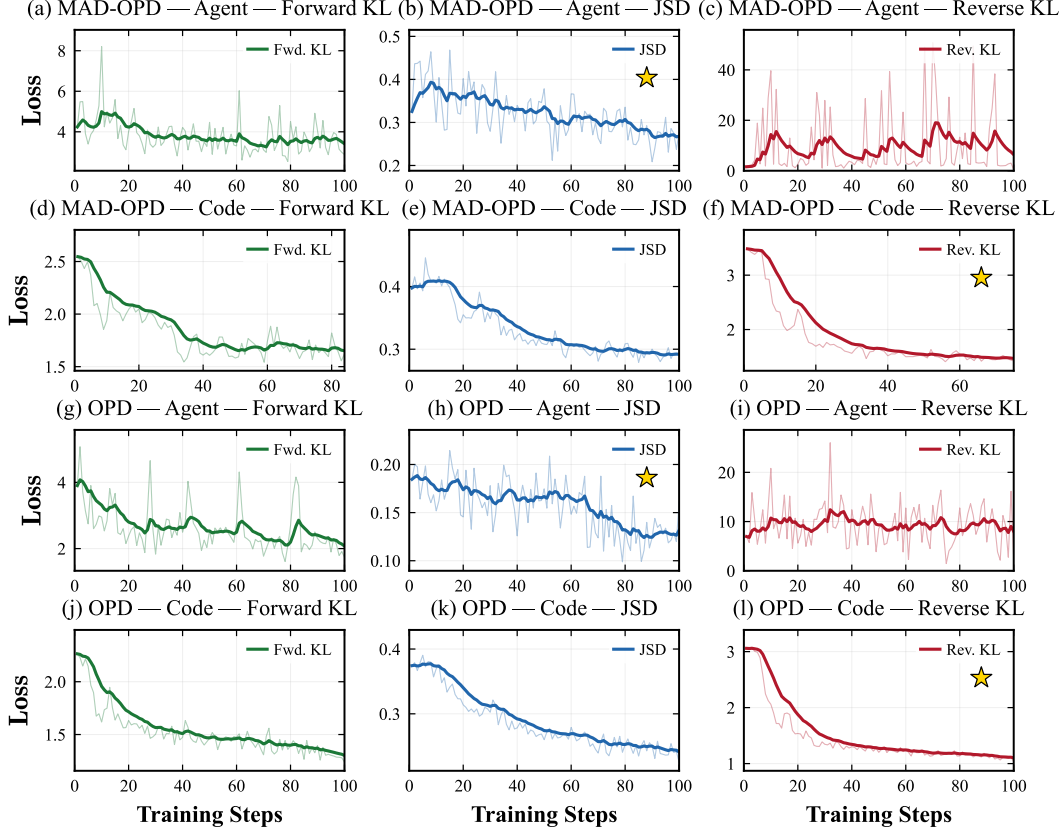


Figure 6: **Training loss curves** for MAD-OPD (rows 1–2) and OPD (rows 3–4) on agentic and code tasks under three divergences (Forward KL, JSD, Reverse KL), 14B+8B→4B. X-axes are gradient steps (gradient accumulation = 8, so absolute loss values are roughly $8\times$ the per-micro-batch value), truncated to the first 100. Each panel overlays the per-step training loss (light) with an exponential moving average (solid) using smoothing factor $\alpha = 0.15$ (i.e. $s_t = 0.15 y_t + 0.85 s_{t-1}$, $s_0 = y_0$). **Agentic panels (rows 1, 3):** reverse KL (column 3) exhibits high-variance spikes consistent with the unbounded per-token logit gradient under privileged p - q gaps (Prop. 1.2), whereas JSD (column 2) remains stable (Prop. 1.1). **Code panels (rows 2, 4):** reverse KL converges smoothly, matching the mode-seeking behavior favored by single-mode code distributions.

Table 6: **Teacher base performance.** Single-model evaluation of the six teachers across all five benchmarks. τ^2 and Vita report the three-domain means defined in App. B.9 / App. B.10. Code metrics: pass@1 with std as subscript over 16 seeds, and BoN@16 with $N=16$ samples per problem. The Qwen3.5 family exhibits a strong baseline on these tasks, leaving smaller per-method headroom for the Qwen3.5 27B+9B→4B configuration discussed in Sec. 5.3.

Teacher	Agentic			Code pass@1		Code BoN@16	
	BFCL-v4	τ^2	Vita	LCB-v6	MBPP+	LCB-v6	MBPP+
<i>Qwen3 family</i>							
Qwen3-8B	28.94	30.08	17.16	20.93 ± 0.76	63.56 ± 0.61	30.86	70.90
Qwen3-14B	32.46	33.70	21.70	25.57 ± 0.87	75.13 ± 0.78	33.14	80.16
Qwen3-30B-A3B [†]	35.72	48.88	24.90	36.00 ± 0.92	77.87 ± 0.81	51.43	80.95
Qwen3-32B	38.01	48.39	25.67	27.43 ± 0.84	78.21 ± 0.65	36.00	81.75
<i>Qwen3.5 family</i>							
Qwen3.5-9B	36.97	49.33	11.72	39.43 ± 1.13	63.36 ± 1.06	55.43	79.37
Qwen3.5-27B	49.04	56.54	11.87	43.43 ± 0.48	72.21 ± 0.97	65.71	81.22

[†] Qwen3-30B-A3B-Instruct-2507 release.

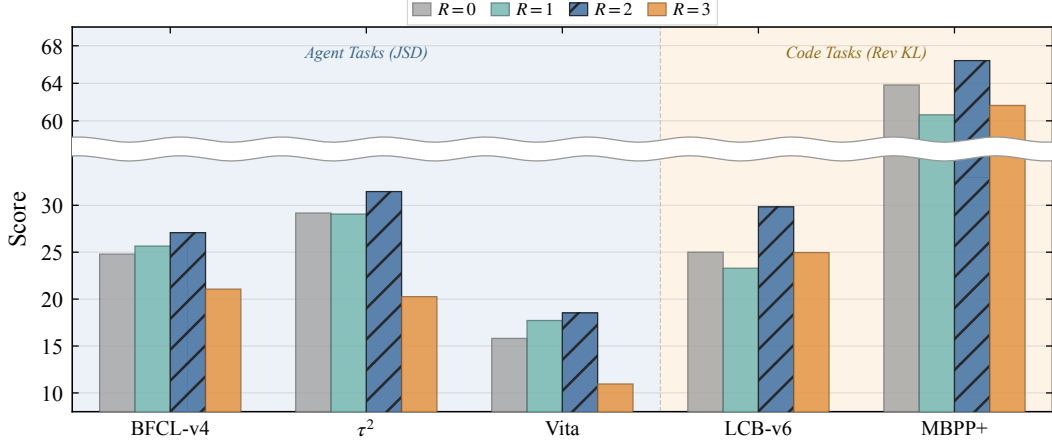


Figure 7: **Effect of debate rounds R** on the 14B+8B→4B configuration. $R=0$: no teacher generation (single-teacher OPD). $R=1$: independent only (MT-OPD). $R=2$ (hatched, default): one inter-teacher revision round. $R=3$ degrades sharply on agentic tasks as the extra round inflates the prompt context. Background shading separates agentic tasks (left, JSD) from code tasks (right, reverse KL). The wavy break compresses the y-axis between 35 and 58 to show both regimes at usable resolution.

D.1 OPD Gradient Derivation

Proof. At each position t , $D(p||q)$ depends on θ only through $q = \pi_\theta(\cdot|x, \hat{y}_{<t})$; the privileged teacher $p = \pi^*(\cdot|x, c, \hat{y}_{<t})$ is a stop-gradient target. Teacher-forcing fixes $\hat{y}_{<t}$ as non-differentiable context, dropping the REINFORCE score-function term $G \cdot \nabla \log \pi$ (with G the trajectory return), so that

$$\nabla_\theta \mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{x, \hat{y} \sim \pi_\theta} \left[\frac{1}{|\hat{y}|} \sum_{t=1}^{|\hat{y}|} \nabla_\theta D(p||q) \right]. \quad (11)$$

As an illustration, decomposing forward KL as $D_{\text{KL}}^\rightarrow(p||q) = H(p, q) - H(p)$ (with $H(p)$ constant in z) and applying the softmax Jacobian $\partial q(v)/\partial z_i = q(v)(\delta_{iv} - q(i))$ collapses the chain rule to

$$\nabla_{z_i} D_{\text{KL}}^\rightarrow(p||q) = - \sum_v p(v)(\delta_{iv} - q(i)) \stackrel{\sum_{p=1}^P}{=} q(i) - p(i), \quad (12)$$

the standard “ $q - p$ ” cross-entropy gradient with $|q(i) - p(i)| \leq 1$ uniformly. Stability thus reduces to the per-position bound $\|\nabla_{z_t} D\|_\infty$, derived per divergence in §D.2–D.4. $\square$

D.2 Proof of Lemma 1 (JSD Bounded Loss and Bounded Logit Gradient)

Restatement. For any distributions p, q over $\mathcal{V}$ and any $\beta \in (0, 1)$, (i) $\text{JSD}_\beta(p||q) \in [0, H(\beta)]$ with $H(\beta) = -\beta \log \beta - (1-\beta) \log(1-\beta)$; (ii) $\|\nabla_z \text{JSD}_\beta(p||q)\|_\infty \leq 2$ uniformly over all p, q .

Proof. Let $m = \beta p + (1-\beta)q$ and $q(v) = \text{softmax}(z)_v$, with $0 \log 0 := 0$; softmax gives $q(v) > 0$, hence $m(v) \geq (1-\beta)q(v) > 0$ everywhere.

Part 1: Loss boundedness. From $m(v) \geq \beta p(v)$ we have $\frac{p(v)}{m(v)} \leq \frac{1}{\beta}$, hence $\log \frac{p(v)}{m(v)} \leq \log \frac{1}{\beta}$, so

$$D_{\text{KL}}(p||m) = \sum_v p(v) \log \frac{p(v)}{m(v)} \leq \sum_v p(v) \log \frac{1}{\beta} = \log \frac{1}{\beta}, \quad (13)$$

and symmetrically $D_{\text{KL}}(q||m) \leq \log \frac{1}{1-\beta}$ from $m(v) \geq (1-\beta)q(v)$. Combining and writing $H(\beta) := -\beta \log \beta - (1-\beta) \log(1-\beta)$,

$$\text{JSD}_\beta(p||q) = \beta D_{\text{KL}}(p||m) + (1-\beta) D_{\text{KL}}(q||m) \leq H(\beta), \quad (14)$$

so $\text{JSD}_{0.5} \leq \log 2$; the lower bound $\text{JSD}_\beta \geq 0$ follows from $D_{\text{KL}} \geq 0$.

Part 2: Gradient boundedness.

Step 1: Decomposition. Logits z enter JSD_β through q only (p fixed); by linearity,

$$\nabla_{z_i} \text{JSD}_\beta = \beta \nabla_{z_i} D_{\text{KL}}(p\|m) + (1-\beta) \nabla_{z_i} D_{\text{KL}}(q\|m), \quad (15)$$

with softmax Jacobian

$$\frac{\partial q(v)}{\partial z_i} = q(v)(\delta_{iv} - q(i)), \quad \frac{\partial m(v)}{\partial z_i} = (1-\beta) q(v)(\delta_{iv} - q(i)). \quad (16)$$

Step 2: $\nabla_{z_i} D_{\text{KL}}(p\|m)$. Only $-\sum_v p(v) \log m(v)$ depends on z . Chain rule with $\nabla_{z_i} \log m(v) = \frac{1}{m(v)} \frac{\partial m(v)}{\partial z_i}$ and (16):

$$\nabla_{z_i} D_{\text{KL}}(p\|m) = -\sum_v \frac{p(v)}{m(v)} \cdot \frac{\partial m(v)}{\partial z_i} = -(1-\beta) \sum_v \frac{p(v) q(v) (\delta_{iv} - q(i))}{m(v)}. \quad (17)$$

Step 3: $\nabla_{z_i} D_{\text{KL}}(q\|m)$. Expanding $D_{\text{KL}}(q\|m) = \sum_v q(v) \log \frac{q(v)}{m(v)}$ and applying the product rule (using $\sum_v \partial q(v)/\partial z_i = 0$):

$$\nabla_{z_i} D_{\text{KL}}(q\|m) = \underbrace{\sum_v q(v)(\delta_{iv} - q(i)) \log \frac{q(v)}{m(v)}}_{=: X} - (1-\beta) \underbrace{\sum_v \frac{q(v)^2 (\delta_{iv} - q(i))}{m(v)}}_{=: Y}. \quad (18)$$

Step 4: Bound on $|\beta \nabla_{z_i} D_{\text{KL}}(p\|m)|$. From (17) with $|\delta_{iv} - q(i)| \leq 1$, then weighted AM-GM ($m \geq p^\beta q^{1-\beta}$) followed by Young's inequality:

$$\frac{p(v)q(v)}{m(v)} \leq p(v)^{1-\beta} q(v)^\beta \leq (1-\beta)p(v) + \beta q(v) \implies \sum_v \frac{p(v)q(v)}{m(v)} \leq 1. \quad (19)$$

Hence

$$|\beta \nabla_{z_i} D_{\text{KL}}(p\|m)| \leq \beta(1-\beta) \leq \frac{1}{4}. \quad (20)$$

Step 5: Bound on $|(1-\beta) \nabla_{z_i} D_{\text{KL}}(q\|m)|$. Splitting the indicator in (18) (the $v = i$ term plus $\sum_{v \neq i}$ collapsed via $\sum_v q(v) = 1$) yields

$$X = q(i) \log \frac{q(i)}{m(i)} - q(i) D_{\text{KL}}(q\|m). \quad (21)$$

For the first term, case-split on $\text{sign} \log \frac{q(i)}{m(i)}$:

$$|q(i) \log \frac{q(i)}{m(i)}| \leq \begin{cases} q(i) \log \frac{1}{1-\beta} \leq \log \frac{1}{1-\beta} & \text{if } q(i) \geq m(i), \\ m(i) - q(i) = \beta(p(i) - q(i)) \leq \beta \leq \log \frac{1}{1-\beta} & \text{if } q(i) \leq m(i), \end{cases} \quad (22)$$

where the first case uses $m(i) \geq (1-\beta)q(i)$ and the second uses $\log x \leq x-1$ together with Taylor expansion. For the second term, $q(i) D_{\text{KL}}(q\|m) \leq D_{\text{KL}}(q\|m) \leq \log \frac{1}{1-\beta}$ by Part 1. Triangle inequality on (21):

$$|X| \leq 2 \log \frac{1}{1-\beta}. \quad (23)$$

For $|Y|$, $q(v)^2/m(v) \leq q(v)/(1-\beta)$ and $|\delta_{iv} - q(i)| \leq 1$ yield $|Y| \leq 1$. Triangle inequality on $X - Y$, then prefactor $(1-\beta)$:

$$|(1-\beta) \nabla_{z_i} D_{\text{KL}}(q\|m)| \leq (1-\beta) \left(2 \log \frac{1}{1-\beta} + 1 \right) \leq \frac{2}{\sqrt{e}}, \quad (24)$$

where $\max_{x \in (0,1)} x(1 - 2 \log x) = 2/\sqrt{e}$ at $x = e^{-1/2}$.

Step 6: Combining. Triangle inequality on (15) via (20), (24):

$$\|\nabla_z \text{JSD}_\beta(p\|q)\|_\infty \leq \frac{1}{4} + \frac{2}{\sqrt{e}} \approx 1.46 < 2, \quad (25)$$

uniformly in $\beta \in (0, 1), p, q$. $\square$

D.3 Proof of Lemma 2 (Reverse KL Mode Concentration)

Restatement. For a mixture $p = \sum_{j=1}^J \alpha_j p_j$ with J well-separated modes (disjoint supports), the reverse-KL minimizer $q^* = \arg \min_q D_{\text{KL}}(q \| p)$ over softmax-parameterized distributions concentrates on the dominant mode p_{j^*} , $j^* = \arg \max_j \alpha_j$, with optimal cost $D_{\text{KL}}(q^* \| p) = -\log \alpha_{j^*}$.

Proof. Let $\{p_j\}$ have pairwise disjoint supports $\mathcal{S}_j \subset \mathcal{V}$ and write the reverse KL in cross-entropy form:

$$D_{\text{KL}}(q \| p) = H(q, p) - H(q), \quad H(q, p) := - \sum_v q(v) \log p(v), \quad (26)$$

where $H(q, p) \geq 0$ is the cross-entropy and $H(q) \geq 0$ the entropy. For $v \notin \bigcup_j \mathcal{S}_j$ we have $p(v) = 0$, so any mass $q(v) > 0$ outside the mode supports makes $H(q, p) = +\infty$; hence $\text{supp}(q^*) \subseteq \bigcup_j \mathcal{S}_j$. Let $\mu_j := \sum_{v \in \mathcal{S}_j} q(v)$ ($\sum_j \mu_j = 1$); within each mode the optimal conditional matches p_j , reducing the objective to

$$D_{\text{KL}}(q^* \| p) = \sum_j \mu_j \log \frac{\mu_j}{\alpha_j} = D_{\text{KL}}(\mu \| \alpha). \quad (27)$$

While an unconstrained categorical distribution could match $\mu = \alpha$ at a single step, the reverse-KL loss is itself weighted by the student’s own probability q : any token with $q(v) \rightarrow 0$ contributes $q(v) \log \frac{q(v)}{p(v)} \rightarrow 0$ (under $0 \log 0 := 0$), so the gradient signal to recover secondary modes vanishes. Once the student slightly favors one mode, optimization self-reinforces toward it; Gu et al. [13] establish this mode-seeking dynamic for LM distillation. The concentrated solution ($\mu_{j^*} \rightarrow 1$ at $j^* = \arg \max_j \alpha_j$) yields

$$D_{\text{KL}}(q^* \| p) = -\log \alpha_{j^*}. \quad (28)$$

□

D.4 Proof of Proposition 1 (Per-Token Gradient Stability)

Restatement. Under the dense OPD gradient estimator (App. D.1), the per-token logit gradient satisfies: (i) JSD: $\|\nabla_{z_t} \text{JSD}_\beta(p \| q)\|_\infty \leq 2$ worst-case, hence $\|\nabla_z \mathcal{L}_{\text{JSD}}(\tau)\|_\infty \leq 2$ irrespective of trajectory length; (ii) Reverse KL: $\|\nabla_{z_t} D_{\text{KL}}^\leftarrow(q \| p)\|_\infty$ is unbounded as $p(i) \rightarrow 0$ with $q(i) > 0$; (iii) Forward KL: $\|\nabla_{z_t} D_{\text{KL}}^\rightarrow(p \| q)\|_\infty \leq 1$ but mode-covering causes a separate failure (App. D.6).

Proof. Throughout this proof, $z = (z_1, \dots, z_N)$ denotes the stacked per-position pre-softmax logit vector, with $z_t \in \mathbb{R}^{|\mathcal{V}|}$. Bounds below are at the logit level; the parameter-level gradient $\nabla_\theta \mathcal{L}$ is the standard sum of per-position contributions through the shared θ .

For a trajectory τ of M steps with $|a_m| \leq T_{\max}$ tokens per action, App. D.1 gives the additive decomposition

$$\nabla_z \mathcal{L}_D(\tau) = \sum_{m=1}^M \sum_{t=1}^{|a_m|} \nabla_{z_t^{(m)}} D(p \| q), \quad (29)$$

with p, q evaluated at the corresponding state. Stability is therefore governed by the worst-case per-token logit gradient $\|\nabla_{z_t} D\|_\infty$.

Part 1: JSD trajectory bound is independent of trajectory length. Lemma 1.2 gives $\|\nabla_{z_t} \text{JSD}_\beta(p \| q)\|_\infty \leq 2$ at every position t . Under the dense OPD estimator, the loss decomposes additively with the t -th term depending on z_t alone:

$$\mathcal{L}_{\text{JSD}}(\tau) = \frac{1}{N} \sum_{t=1}^N \text{JSD}_\beta(p_t \| q_t), \quad N := \sum_{m=1}^M |a_m|. \quad (30)$$

Per-position partial gradients are decoupled (Hessian block-diagonal across positions); the ℓ_∞ norm picks the maximum over blocks rather than summing them:

$$\|\nabla_z \mathcal{L}_{\text{JSD}}(\tau)\|_\infty = \frac{1}{N} \max_{1 \leq t \leq N} \|\nabla_{z_t} \text{JSD}_\beta(p_t \| q_t)\|_\infty \leq \frac{2}{N} \leq 2. \quad (31)$$

Two consequences. **(a) Per-position uniformity:** the max-of-blocks structure ensures the worst-case magnitude is determined by the per-position bound rather than by trajectory length; (31) holds at the

per-position level for any N and any teacher–student gap. **(b) Robust under both reductions:** mean reduction (Eq. 1) sharpens to $2/N$; sum reduction stays at 2; the OPAD loss (Eq. 10, sum-of-per-step-means) inherits each block’s $(1/|a_m|) \cdot 2 \leq 2$, giving $\|\nabla_z \mathcal{L}_{\text{OPAD}}\|_\infty \leq 2$. This per-position guarantee fails for reverse KL (Part 2), where the unbounded per-position gradient is not tamed by any reduction. The per-position bound for JSD is what underlies (31).

Part 2: Reverse KL is unbounded under privileged p – q gaps. By Eq. 3, $D_{\text{KL}}^{\leftarrow}(p\|q) = D_{\text{KL}}(q\|p)$. Direct computation via the softmax Jacobian gives

$$\begin{aligned} \nabla_{z_i} D_{\text{KL}}^{\leftarrow}(p\|q) &= \sum_v \frac{\partial q(v)}{\partial z_i} \left[\log \frac{q(v)}{p(v)} + 1 \right] - \underbrace{\sum_v \frac{\partial q(v)}{\partial z_i}}_{=0} \\ &= \sum_v q(v)(\delta_{iv} - q(i)) \log \frac{q(v)}{p(v)}. \end{aligned} \quad (32)$$

The diagonal contribution $q(i)(1 - q(i)) \log(q(i)/p(i))$ diverges to $+\infty$ as $p(i) \rightarrow 0$ with $q(i) > 0$. Privileged distillation creates exactly this regime whenever the student samples a token the privileged teacher deems unlikely, producing the gradient spikes empirically observed in Figure 6. No finite trajectory-level bound on $\|\nabla_z \mathcal{L}_{\text{KL}}^{\leftarrow}(\tau)\|_\infty$ exists.

Part 3: Forward KL is bounded but mode-covering. Forward KL reduces to the cross-entropy gradient

$$\nabla_{z_i} D_{\text{KL}}^{\rightarrow}(p\|q) = q(i) - p(i) \in [-1, 1], \quad (33)$$

so $\|\nabla_{z_i} D_{\text{KL}}^{\rightarrow}\|_\infty \leq 1$ uniformly. Its agentic failure mode is geometric (mode-covering across multi-step rollouts), analyzed in App. D.6. $\square$

D.5 Proof of Proposition 2 (Reverse KL for Coherent Code Generation)

Restatement. For a multi-modal code distribution $p = \sum_{j=1}^J \alpha_j p_j$ with well-separated modes and a softmax-parameterized student q^* minimizing each divergence: (i) under reverse KL, q^* concentrates on the dominant mode p_{j^*} ; (ii) under forward KL, q^* covers every mode; (iii) under JSD, q^* partially concentrates but retains non-negligible mass on secondary modes.

Proof. Let each p_j represent a distinct correct implementation with support $\mathcal{S}_j$ ($j = 1, \dots, J$), with $\mathcal{S}_i \cap \mathcal{S}_j = \emptyset$ for $i \neq j$.

Part 1: Reverse KL concentrates on the dominant mode. By Lemma 2, the softmax-parameterized minimizer satisfies

$$q^*(\cdot \mid \text{prefix}) \approx p_{j^*}(\cdot \mid \text{prefix}), \quad j^* = \arg \max_j \alpha_j, \quad (34)$$

assigning near-zero probability to tokens of other modes, preventing splices such as switching mid-function from a recursive to an iterative implementation.

Part 2: Forward KL covers all modes. $D_{\text{KL}}^{\rightarrow}(p\|q) = \sum_v p(v) \log(p(v)/q(v)) = \infty$ whenever $q(v) \rightarrow 0$ on $\text{supp}(p)$, so the minimizer must spread mass across every mode:

$$q_{\rightarrow}^*(v) = p(v) = \sum_j \alpha_j p_j(v), \quad \forall v \in \mathcal{V}. \quad (35)$$

Sequential token sampling under (35) interpolates across implementation paths, producing incoherent code.

Part 3: JSD retains secondary-mode mass. The mixture $m = \beta p + (1-\beta)q$ removes the infinite inter-mode penalty of reverse KL. For $\beta = 0.5$,

$$\nabla_q \text{JSD}_{0.5}(p\|q) = \frac{1}{2} \sum_v \nabla_q [q(v) \log \frac{q(v)}{m(v)}], \quad (36)$$

and $m(v) \geq 0.5 p(v) > 0$ on every mode support means the gradient does not force q to abandon secondary modes: adequate when multiple action sequences are valid (agentic), suboptimal when single-implementation commitment is required (code). $\square$

D.6 Supplementary: Forward KL Mode-Covering Instability

By (33), the forward-KL per-logit gradient satisfies $\nabla_{z_i} D_{\text{KL}}^{\rightarrow}(p||q) = q(i) - p(i) \in [-1, 1]$, so it does not produce gradient spikes from privileged p - q gaps. Its agentic underperformance is geometric, not numerical.

In multi-step tool use, multiple action sequences typically attain the same goal (e.g., calling tool A vs. tool B), making the debate-enriched teacher distribution p multi-modal at agentic decision points. Forward KL imposes an infinite penalty on missing modes ($p(v) > 0 \implies q(v) > 0$), pulling the student to place non-trivial mass on every viable strategy simultaneously. Sequential token sampling then mixes incompatible strategies (e.g., the tool name from strategy A paired with the argument schema of strategy B), producing invalid actions that derail long trajectories. These compounding step-level failures explain forward KL’s consistently poor agentic performance in Table 4.

E Case Study: Emergent Error Correction via Multi-Agent Debate

Complementing the schematic in Figure 1, we present an *illustrative* (rather than calibration) worked-out example on a different task domain from the ToolACE training set [26], where a single 14B teacher fails, but a two-round debate between the 14B and 8B teachers yields the correct answer. We show the complete debate trajectory (each teacher’s brief inline analysis and function-call output) to illustrate how complementary errors are corrected through deliberation; the population-level robustness of the confidence-weighted aggregation is discussed in App. C.1. *For readability, the inline analysis text below is lightly paraphrased from the model’s actual output, while the function-call arguments and confidence scores are reproduced verbatim; the underlying complementary-error pattern is consistently observed across the ToolACE training set, not cherry-picked.*

Problem: ToolACE Training-Set Tool-Calling Task

System: Current time is 2020-10-06 04:12:36. Two functions available: `travel_budget_planner` (requires destination, start/end dates, accommodation, activities) and `travel_insurance_quote_generator` (requires age, countries, trip duration, coverage type, deductible).

User: “I plan on taking a trip **next month** from the **16th to the 30th**. I’ll be visiting Japan and I prefer staying in a hotel. Some of my preferred activities would include sightseeing and local cuisine tasting. Can you help me with a budget plan for this trip? Also, can you get me an insurance quote for this trip? I am 34 with no medical conditions and I’d like an all-inclusive insurance package with a deductible of \$200.”

Key reasoning challenges: (1) “next month” from Oct 6 = **November** 2020; (2) 16th to 30th = **14 days**; (3) both functions must be called with correctly nested parameters.

Ground Truth

```
[travel_budget_planner(travel_details=[{"destination": "Japan",
  "dates": {"start_date": "2020-11-16", "end_date": "2020-11-30"},
  "preferences": {"accommodation": "hotel", "activities": ["sightseeing",
    "local cuisine tasting"]}}]),
travel_insurance_quote_generator(insurance_details={"personal_info": {"age":
34},
  "trip_info": {"countries_visited": ["Japan"], "trip_duration": 14},
  "coverage_preferences": {"coverage_type": "all_inclusive", "deductible":
200}})]
```

Setting 1: Single 14B Teacher**✗ Incorrect***Model output (paraphrased):*

“The user wants to plan a trip and get an insurance quote. Destination: Japan. Dates: from the 16th to the 30th of next month. Since the current time is 2020-10-06, **the trip dates would be October 16 to October 30**. Accommodation: hotel. Trip duration: Oct 16 to Oct 30 = 14 days.”

Function call (key arguments):

start_date: "2020-10-16" ← wrong month
 end_date: "2020-10-30" ← wrong month
 trip_duration: 14 ✓

*Confidence: c = 90***Round 1: Expert A (14B Teacher)****✗ Incorrect***Model output (paraphrased):*

“Dates: next month, 16th to 30th. Current date is October 6, 2020. **So the dates are October 16 to October 30, 2020**. Duration: 16th to 30th = 14 days.”

Function call (key arguments):

start_date: "2020-10-16" ← wrong month
 end_date: "2020-10-30" ← wrong month
 trip_duration: 14 ✓

*Confidence: c = 85***Round 1: Expert B (8B Teacher)****✗ Incorrect***Model output (paraphrased):*

“The user wants to visit Japan. They said ‘next month from the 16th to the 30th’. Current time is 2020-10-06, **so next month is November**. Start date: 2020-11-16, End date: 2020-11-30. Trip duration: **30 – 16 = 15 days** (counting both endpoints).”

Function call (key arguments):

start_date: "2020-11-16" ✓
 end_date: "2020-11-30" ✓
 trip_duration: 15 ← wrong duration

Confidence: c = 80

Key observation: Errors are **complementary**: Expert A gets the month wrong but the duration right, while Expert B gets the month right but the duration wrong.

Round 2: Expert A (14B Teacher)**✓ Correct***Model output (paraphrased, after seeing Expert B’s Round 1 answer):*

“Looking at Expert B’s answer, they used **November** dates while I used October dates. Let me reconsider. The user said ‘next month’ and the current date is October 6. **‘Next month’ means the month after October, which is November. Expert B is correct on this point; I made an error.**

However, Expert B calculated trip duration as 15 days. From November 16 to November 30, the duration is **30 – 16 = 14 days**. My original duration of 14 was correct.

Let me revise: use the correct November dates while keeping the correct trip duration of 14 days.”

Function call (key arguments):

start_date: "2020-11-16" ✓ corrected
 end_date: "2020-11-30" ✓ corrected
 trip_duration: 14 ✓ retained

Confidence: c = 95 (↑ from 85)

Table 7: **Debate progression.** Green = correct, red = incorrect (also indicated by $\checkmark/\times$ markers for colorblind readers). Complementary Round 1 errors cancel after one debate round; both teachers converge to the ground truth with differentiated confidence.

Setting	Teacher	start_date	end_date	trip_dur.	Conf.
Single teacher	14B	10-16	10-30	14	90
Round 1	14B	10-16	10-30	14	85
	8B	11-16	11-30	15	80
Round 2	14B	11-16	11-30	14	95
	8B	11-16	11-30	14	90
Ground truth	–	11-16	11-30	14	–

Round 2: Expert B (8B Teacher)

$\checkmark$ Correct

Model output (paraphrased, after seeing Expert A’s Round 1 answer):

“Expert A used October dates while I used November. The user said ‘next month’ and the current time is October 6, so next month is November. **I believe my date calculation is correct.**

However, Expert A used trip_duration of 14 while I used 15. Let me reconsider. From November 16 to November 30: $30 - 16 = 14$ days. **In travel planning, the standard convention is to count the difference between end and start dates. So 14 is the more appropriate value.**”

Function call (key arguments):

start_date: "2020-11-16" $\checkmark$ retained
end_date: "2020-11-30" $\checkmark$ retained
trip_duration: 14 $\checkmark$ corrected

Confidence: $c = 90$ ($\uparrow$ from 80)

Summary. Table 7 summarizes the debate progression. The 14B teacher’s confidence drops from $c=90$ as a single teacher to $c=85$ in Round 1 because the debate prompt explicitly signals that another teacher will weigh in, prompting more cautious self-assessment; after the Round 2 revision corrects its error, confidence rises to $c=95$. The key mechanism is *selective adoption*: each teacher identifies and corrects its own error by adopting the other teacher’s correct component, while retaining what it already had right. This produces differentiated confidence weights ($w_{14B} = e^{0.95}/(e^{0.95}+e^{0.90}) \approx 0.513$, $w_{8B} \approx 0.487$ via Eq. 7), reflecting the 14B teacher’s stronger overall reasoning ability.

Token-level supervision quality. Figure 8 visualizes the per-token divergence $D(p_{T_k}||p_S)$ when teachers force-decode the student’s on-policy start_date argument. Under the single 14B teacher, all tokens receive low divergence because the teacher shares the student’s wrong month; no corrective gradient exists. Under MAD-OPD Round 2, the debate-corrected teachers concentrate high divergence on the erroneous month token -10, producing a strong corrective signal toward -11.

Implications for distillation. The debate trace from Round 2 provides richer training signal than either teacher’s Round 1 output alone. The student learns not just the correct answer but, through the privileged force-decoding mechanism, the *reasoning pattern of error detection and self-correction*. This emergent collective intelligence, where the whole exceeds the sum of its parts, is the central advantage of MAD-OPD over single-teacher OPD.

F Debate Prompt Templates

This appendix lists the prompt templates used in each debate round. We adopt a two-round protocol: **Round 1** elicits independent responses, and **Round 2** presents both responses as debate context for revision. Templates are instantiated per task domain (agent / code); placeholders are shown in {braces}. When confidence-based weighting is enabled (§4.2), every prompt is appended with the confidence suffix shown in §F.3.

Student on-policy output (start_date argument, tokenized):

" start_date ": " 2020 -10 -16 "

Ground truth: -11 (November); student writes -10 (October).

Per-token divergence $D(p_T \| p_S)$:

Supervision	Token-level divergence heatmap
Single 14B	" start_date ": " 2020 -10 -16 " <i>Uniformly low: teacher has the same error, no corrective signal.</i>
MAD-OPD Round 2	" start_date ": " 2020 -10 -16 " <i>High divergence on month token: teachers signal "-11" (November).</i>

Figure 8: **Token-level supervision heatmap** for the start_date argument. Color intensity encodes divergence: low (agreement), med, high (strong corrective gradient). The single 14B teacher shares the student’s wrong month, providing zero corrective signal. After debate, MAD-OPD teachers concentrate supervision on the erroneous token.

F.1 Round 1: Independent Response

Each teacher receives only the original input x and generates an initial response without seeing the other teacher’s output. The system instruction is domain-specific:

Round 1: Agentic Tasks

You are an expert in AI tool-use and function calling. Please analyze the following user request and provide the best tool-use response.

User request: {prompt}

Your response:

Round 1: Code Generation

You are an expert programmer specializing in competitive programming and code golf. Please solve the following programming challenge. Focus on algorithm correctness, edge case handling, and code efficiency.

Problem:

{prompt}

Your code solution:

F.2 Round 2: Debate with Shared History

Each teacher receives the original input together with both Round 1 responses, and is asked to revise its answer after considering the other expert’s perspective. The prompt explicitly states “*You don’t have to agree*”, encouraging genuine deliberation rather than conformity.

Round 2: Agentic Tasks (shown for Expert A; Expert B is symmetric)

{prompt}

Two experts have provided their tool-use responses to this request:

[Expert A, Round 1]:

{teacher_1_answer}

[Expert B, Round 1]:

{teacher_2_answer}

You are Expert A. You don’t have to agree with the other expert’s response. Please continue the debate about the best tool-use strategy. Having seen the other expert’s response, please reconsider and provide your improved response focusing on tool selection accuracy, parameter correctness, and call sequence optimization.

Round 2: Code Generation (shown for Expert A)

{prompt}

Two expert programmers have provided their code solutions to this challenge:

[Expert A, Round 1]:
{teacher_1_answer}

[Expert B, Round 1]:
{teacher_2_answer}

You are Expert A. You don't have to agree with the other expert's solution. Please continue the debate about the best approach. Having seen the other expert's code, please reconsider and provide your improved solution focusing on algorithm correctness, edge case handling, time/space complexity, and code conciseness.

F.3 Confidence Scoring Suffix

When confidence-based dynamic weighting is enabled, the following line is appended to every prompt (both rounds). The confidence scores c_k extracted from the final round are normalized into teacher weights w_k as described in Eq. 6–7.

Confidence suffix (appended to all prompts when `dynamic_weights=True`)

End your response with a confidence score line: Confidence Score: N (where N is 0-100)

After the debate, the $K \cdot R = 4$ responses are concatenated into the debate transcript $\mathcal{H}_m^R$ that serves as privileged context during force-decoding (§4.3):

Debate transcript structure

[Debate Start]
[Expert A Initial Answer]: {round1_teacher1}
[Expert B Initial Answer]: {round1_teacher2}
[Expert A Revised Answer]: {round2_teacher1}
[Expert B Revised Answer]: {round2_teacher2}
[Debate End]

G Broader Impact

MAD-OPD is a training-efficiency method whose primary impact is to reduce the parameter count required to attain a given level of agentic and code performance, thereby lowering the GPU and energy footprint of deploying capable language-model assistants. As a concrete instance (App. C.3), a 4B student trained under our 14B+8B teacher debate exceeds its 14B teacher on LCB-v6 at competitive token cost, suggesting that small models distilled by MAD-OPD can replace much larger teachers for inference. This makes capable assistants more accessible to research groups and downstream users with limited compute, lowers the carbon cost of large-scale serving, and shifts more of the cost from the inference path to a one-time distillation step. Because MAD-OPD introduces no new capability beyond the teacher pool, the distilled student inherits the safety properties and refusal behaviour of its base Qwen3 / Qwen3.5 checkpoints; the only residual risk is that any capability the teachers already possess becomes cheaper to deploy. We mitigate this by using only publicly released open-weight models, not training on safety-relevant red-team data, and releasing checkpoints under the same license terms as the base models, leaving application-level safety filtering to downstream deployers as is standard for open-weight LLMs.